

CausalMech-Bio

A Complete Guide to Causal Mechanistic Circuit Discovery in Protein
Language Models

September 2026

Contents

CausalMech-Bio — A Complete Guide	4
How to read this document	4
Table of contents	4
Part 0 — What this project does, in plain language	6
0.1 The setting	6
0.2 The model	6
0.3 The question	6
0.4 Why the usual answer is not good enough	7
0.5 What this project does instead	7
0.6 What comes out	7
0.7 What it does not do	8
Part 1 — The biology	9
1.1 Proteins and amino acids	9
1.2 Sequence and structure	10
1.3 What proteins do, and where	11
1.4 Mutations	11
1.5 Measuring what a mutation does	12
1.6 Where 3D structures come from	13
Part 2 — The machine learning	15
2.1 What a neural network is	15
2.2 Training: loss, gradients, backpropagation	16
2.3 Optimisers and schedules	17
2.4 Language models and the masked objective	17
2.5 Tokens, vocabularies and embeddings	18
2.6 Attention	19
2.7 The transformer block	19
2.8 The residual stream	20
2.9 ESM-2	21
2.10 Scoring a mutation with ESM-2	22
Part 3 — Interpretability: what goes wrong	24
3.1 What we want from an explanation	24
3.2 Post-hoc saliency	24
3.3 Why attention is not explanation	25
3.4 Polysemantic neurons	25
3.5 Superposition	26
3.6 Features as directions	27
Part 4 — Sparse autoencoders	28
4.1 Autoencoders	28
4.2 Dictionary learning	28
4.3 The SAE recipe	28
4.4 L1 sparsity and its problems	29
4.5 Top-K	30

4.6 The three details that decide whether it works	31
4.7 Activation normalisation	33
4.8 Measuring a dictionary	33
4.9 Measuring a single feature	34
Part 5 — Causality and path patching	35
5.1 Correlation and causation inside a network	35
5.2 Interventions and the do-operator	35
5.3 Activation patching	36
5.4 Choosing the metric	36
5.5 Direct causal effect	37
5.6 Patching features, not neurons	38
5.7 Error-preserving splicing ▲	39
5.8 The hook-leak bug	40
5.9 Ablation	41
5.10 Attribution patching ▲	41
5.11 From features to circuits	43
5.12 Greedy selection	43
5.13 What conservation means	44
Part 6 — Grounding circuits in structure	46
6.1 The question	46
6.2 Why sequence position $\neq$ structure residue	46
6.3 Sequence alignment ▲	47
6.4 Geometry	48
6.5 Permutation tests ▲	49
6.6 Secondary structure	50
Part 7 — Benchmarking	52
7.1 What a good method must do	52
7.2 Raw attention	52
7.3 Integrated gradients ▲	53
7.4 Random	53
7.5 Causal precision	54
7.6 Faithfulness	54
Part 8 — The codebase, file by file	56
8.1 src/utils/ — foundations	56
8.2 src/models/ — the model and the dictionary	57
8.3 src/interpretability/ — the method	58
8.4 src/validation/ — checking the answer	59
8.5 scripts/	60
8.6 dashboard/app.py (372 lines)	60
8.7 tests/ (1,406 lines, 79 tests)	60
Part 9 — Running it	61
9.1 Install	61
9.2 Verify the install — 1 minute, no downloads	61
9.3 Fetch assets — 30-90 minutes, depending on connection	62
9.4 Train the autoencoders — 15-20 min per layer (8M model)	62
9.5 Discover a circuit — 30-60 seconds per variant	62
9.6 Benchmark — ~20 minutes for 25 variants	63
9.7 Inspect — interactive	63
9.8 A complete first session	63
Part 10 — Reading the results	65
10.1 Was there an effect to explain?	65
10.2 Does the circuit hold together?	65
10.3 Is the circuit spatially real?	66
10.4 Is it better than the free alternatives?	66

10.5 Warning signs	66
10.6 The comparison worth running once	67
Part 11 — Limitations and what to do next	68
11.1 Limitations	68
11.2 What to do next, in order	69
Appendix A — Glossary	70
Appendix B — Index conventions	74
Appendix C — Notation	75

CausalMech-Bio — A Complete Guide

Causal mechanistic circuit discovery in protein language models.

Written for a reader with no prior background. Every term is defined the first time it appears. Nothing is assumed except the ability to read code.

How to read this document

The guide is long because it builds from the ground up. You do not have to read it in order.

- **If you want to understand what the project is for**, read Part 0 and stop. It is two pages and assumes nothing.
- **If you know machine learning but not biology**, start at Part 1, skip Part 2, resume at Part 3.
- **If you know biology but not machine learning**, skip Part 1 and start at Part 2.
- **If you want to run the code**, Part 9 is self-contained; come back for the concepts when a term confuses you.
- **If you hit an unfamiliar word anywhere**, Appendix A is a glossary of every term this document defines, in alphabetical order.

Sections marked **▲ Advanced** can be skipped on a first pass. They contain the derivations and the design arguments, not new facts you need for the rest.

Throughout, boxes marked **Why this and not the obvious thing** explain a design decision where the straightforward implementation would be wrong. These are the intellectual content of the project. If you read nothing else technical, read those.

Table of contents

Part 0 — What this project does, in plain language

Part 1 — The biology 1.1 Proteins and amino acids · 1.2 Sequence and structure · 1.3 What proteins do, and where · 1.4 Mutations · 1.5 Measuring what a mutation does · 1.6 Where 3D structures come from

Part 2 — The machine learning 2.1 What a neural network is · 2.2 Training: loss, gradients, backpropagation · 2.3 Optimisers and schedules · 2.4 Language models and the

masked objective · 2.5 Tokens, vocabularies and embeddings · 2.6 Attention · 2.7 The transformer block · 2.8 The residual stream · 2.9 ESM-2 · 2.10 Scoring a mutation with ESM-2

Part 3 — Interpretability: what goes wrong 3.1 What we want from an explanation · 3.2 Post-hoc saliency · 3.3 Why attention is not explanation · 3.4 Polysemantic neurons · 3.5 Superposition · 3.6 Features as directions

Part 4 — Sparse autoencoders 4.1 Autoencoders · 4.2 Dictionary learning · 4.3 The SAE recipe · 4.4 L1 sparsity and its problems · 4.5 Top-K · 4.6 The three details that decide whether it works · 4.7 Activation normalisation · 4.8 Measuring a dictionary · 4.9 Measuring a single feature

Part 5 — Causality and path patching 5.1 Correlation and causation inside a network · 5.2 Interventions and the do-operator · 5.3 Activation patching · 5.4 Choosing the metric · 5.5 Direct causal effect · 5.6 Patching features, not neurons · 5.7 Error-preserving splicing ▲ · 5.8 The hook-leak bug · 5.9 Ablation · 5.10 Attribution patching ▲ · 5.11 From features to circuits · 5.12 Greedy selection · 5.13 What conservation means

Part 6 — Grounding circuits in structure 6.1 The question · 6.2 Why sequence position $\neq$ structure residue · 6.3 Sequence alignment ▲ · 6.4 Geometry · 6.5 Permutation tests ▲ · 6.6 Secondary structure

Part 7 — Benchmarking 7.1 What a good method must do · 7.2 Raw attention · 7.3 Integrated gradients ▲ · 7.4 Random · 7.5 Causal precision · 7.6 Faithfulness

Part 8 — The codebase, file by file

Part 9 — Running it

Part 10 — Reading the results

Part 11 — Limitations and what to do next

Appendix A — Glossary Appendix B — Index conventions Appendix C — Notation

Part 0 — What this project does, in plain language

0.1 The setting

A **protein** is a molecule your cells build to do a job — digest a sugar, carry oxygen, copy DNA. Each protein is built from a chain of small units called **amino acids**, and that chain then folds into a specific three-dimensional shape. The shape is what does the work.

There are twenty common amino acids, so a protein can be written as a string of letters, one per amino acid, like this:

MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEK . . .

If you change one letter in that string, you have a **mutation**. Sometimes nothing happens. Sometimes the protein stops working, and a person gets sick. Predicting which is which is a major open problem in biology and medicine.

0.2 The model

A **protein language model** is a neural network trained on hundreds of millions of protein sequences, in much the same way that a text language model is trained on sentences. It is shown sequences with letters hidden and learns to guess what was hidden. **ESM-2**, made by Meta, is the best-known open example, and it is the model this project studies.

After that training, ESM-2 turns out to be surprisingly good at judging mutations. Ask it “how surprised are you to see a T here instead of an A?” and its answer correlates with whether the mutation actually breaks the protein in the lab. Nobody taught it that. It learned something about protein function from sequences alone.

0.3 The question

What did it learn?

This is not idle curiosity. If ESM-2 is going to be used to triage patient variants or design enzymes, we need to know whether it has internalised something about protein chemistry or has instead memorised statistical quirks of the databases it was trained on. Those two possibilities produce identical predictions on data that resembles the training set, and diverge exactly where it matters — on the unusual cases.

0.4 Why the usual answer is not good enough

The standard way to “explain” a model like this is to draw a heatmap. Show which residues the model **attended to** when it made its prediction, colour them red, and observe that they look meaningful.

The problem is that attending to something is not the same as using it. A model can look hard at a residue and do nothing with what it saw. Heatmaps of this kind are produced by *observing* the model, and observation cannot distinguish “this mattered” from “this was merely present”. The scientific term for the gap is the difference between **correlation** and **causation**, and it is the same gap that separates “people who carry lighters get lung cancer” from “lighters cause lung cancer”.

0.5 What this project does instead

Three moves, in order.

Move one: find the right units. Individual neurons in a network like ESM-2 are not interpretable — a single neuron fires for several unrelated things at once. There is a mathematical reason for this (Part 3.5) and a known remedy: fit a second, much wider network called a **sparse autoencoder** whose job is to re-express the model’s internal state as a combination of a few items drawn from a large learned vocabulary. Those items tend to be individually meaningful in a way neurons are not. We train one of these on ESM-2’s internal state.

Move two: intervene, do not observe. Take a wild-type protein and its mutant. Run both through ESM-2. Then, in the middle of the mutant’s forward pass, reach in and overwrite one sparse-autoencoder feature with the value it had in the wild-type run, and let the rest of the computation continue. If the model’s judgement about the mutation snaps back toward its wild-type judgement, that feature was *causally responsible*. If nothing happens, it was a bystander, however brightly it lit up on a heatmap.

This is the same logic as a controlled experiment: change one thing, hold everything else fixed, measure the outcome. In the interpretability literature it is called **activation patching** or **path patching**.

Move three: check the answer against physical reality. A circuit is a set of features, and each feature acts at particular positions in the sequence — that is, at particular amino acids. We can look up where those amino acids sit in the protein’s actual 3D structure, which has been measured experimentally and deposited in a public database. If the residues a circuit relies on turn out to be clustered together in space — forming a pocket, an active site, a real physical feature — then the model has found something about the protein’s chemistry. If they are scattered randomly through the structure, it has found a statistical shortcut.

This third move is what distinguishes the project from most interpretability work. Usually there is no ground truth to check an explanation against. In structural biology, there is.

0.6 What comes out

For a given protein and mutation, the pipeline produces:

- a small set of **features** (typically three to eight) at specific layers of the model, with a number saying how much of the mutation's effect each one explains;
- the **residues** each feature acts on;
- a statistical test of whether those residues are more tightly clustered in 3D than you would expect from a random set of the same size;
- a comparison against the standard heatmap methods on two measures that require actually intervening on the model rather than just ranking residues;
- an interactive 3D viewer to look at all of it.

0.7 What it does not do

It does not tell you whether a mutation is pathogenic — that is ESM-2's job, and ESM-2 is only moderately good at it. This project explains *how ESM-2 arrives at its answer*, which is a different and more modest question.

It also does not, yet, establish that a circuit found for one mutation generalises to other mutations in the same protein. Every circuit here is a case study. Turning case studies into a finding is the obvious next experiment and is discussed in Part 11.

Part 1 — The biology

Everything in this part is standard molecular biology. If you know it, skip to Part 2. Nothing here is specific to the project; it is the vocabulary the rest of the document uses.

1.1 Proteins and amino acids

The chain

An **amino acid** is a small molecule with a common backbone and a variable part called a **side chain** (or **R group**). The backbone is identical in all of them; the side chain is what makes glycine different from tryptophan.

Amino acids link end to end into a chain. The bond joining them is a **peptide bond**, and a chain of them is a **polypeptide**. A **protein** is one or more polypeptide chains folded into a working shape.

Twenty amino acids are used by essentially all life. Each has a three-letter code and a one-letter code:

Letter	Three-letter	Name	Side chain character
A	Ala	Alanine	small, hydrophobic
C	Cys	Cysteine	can form disulfide bridges
D	Asp	Aspartate	negatively charged
E	Glu	Glutamate	negatively charged
F	Phe	Phenylalanine	large, aromatic, hydrophobic
G	Gly	Glycine	smallest — just a hydrogen; very flexible
H	His	Histidine	can be charged or not; common in active sites
I	Ile	Isoleucine	hydrophobic, branched
K	Lys	Lysine	positively charged
L	Leu	Leucine	hydrophobic, branched
M	Met	Methionine	hydrophobic; usually starts the chain
N	Asn	Asparagine	polar, uncharged
P	Pro	Proline	rigid; breaks helices
Q	Gln	Glutamine	polar, uncharged
R	Arg	Arginine	positively charged
S	Ser	Serine	small, polar
T	Thr	Threonine	small, polar
V	Val	Valine	hydrophobic, branched
W	Trp	Tryptophan	largest; aromatic

Letter	Three-letter	Name	Side chain character
Y	Tyr	Tyrosine	aromatic, polar

Hydrophobic means “water-avoiding”. **Polar** and **charged** mean “water-friendly”. This single distinction drives most of protein folding: in the watery interior of a cell, a chain folds so that hydrophobic side chains are buried inside and polar ones face out. That is not a rule someone imposed; it is what minimises free energy.

In code, `src/utils/protein.py` defines these twenty letters as `AA_ALPHABET`:

```
AA_ALPHABET = "ACDEFGHIKLMNPQRSTVWY"
```

and a mapping from three-letter PDB codes back to one letter in `three_to_one`.

Non-standard letters

Sequence databases contain a few extra characters, and ESM-2’s vocabulary includes them:

- **X** — unknown or unspecified residue.
- **B** — either aspartate (D) or asparagine (N); the experiment could not tell.
- **Z** — either glutamate (E) or glutamine (Q).
- **U** — selenocysteine, a genuine 21st amino acid, rare.
- **O** — pyrrolysine, a genuine 22nd amino acid, rarer.

The function `validate_sequence(sequence, allow_noncanonical=True)` accepts these by default and rejects anything else. Set `allow_noncanonical=False` when you need a clean 20-letter sequence.

1.2 Sequence and structure

Proteins are described at four levels. You need the first two and should recognise the third.

Primary structure is the sequence of amino acids, written as a string of letters from the N-terminus (the start) to the C-terminus (the end). This is what a protein language model reads.

Secondary structure is local, repeating shape formed by hydrogen bonds along the backbone. Two forms dominate:

- the **α -helix**, a right-handed coil with about 3.6 residues per turn;
- the **β -sheet**, made of extended strands lying side by side.

Everything else is called **loop** or **coil**, which is not to say it is unstructured — loops often carry the residues that do the actual chemistry.

Tertiary structure is the full 3D shape of one chain: where every atom sits. This is what gets measured experimentally and deposited in the PDB (Section 1.6).

Quaternary structure is how multiple chains assemble. Haemoglobin, for example, is four chains together. This project works one chain at a time.

Why the shape matters more than the sequence

Two residues that are 50 positions apart in the sequence can be 4 Å apart in space, touching each other, jointly forming a pocket that binds a molecule. The sequence gives no hint of this. It is the folded shape that determines function, and the sequence matters only because it determines the shape.

This is the crux of the project's validation step. If ESM-2 has learned real protein chemistry, the residues it relies on should be neighbours *in space* even when they are far apart *in sequence*. That is a checkable prediction.

1.3 What proteins do, and where

A protein's job usually happens at a small part of it.

An **active site** is the set of residues where an enzyme performs its chemical reaction. It is typically a handful of residues — often three to six — brought together by folding from scattered parts of the sequence.

A **binding site** or **pocket** is a cavity where another molecule docks.

A **catalytic triad** is a classic example: three residues (commonly serine, histidine and aspartate) positioned within a few ångströms of one another, which together do chemistry that none could do alone. In the sequence they might be at positions 195, 57 and 102 — nowhere near each other.

An **ångström** (Å) is 10^{-10} metres. For scale: a carbon-carbon bond is about 1.5 Å, a hydrogen bond about 3 Å, and two residues are generally considered to be “in contact” if their α -carbons are within 6–8 Å. The project's default distance threshold of 6.0 Å comes from this convention.

Mutating a residue in an active site usually destroys function. Mutating one on the surface, far from anything, usually does nothing. This is the signal the whole project is built on.

1.4 Mutations

Point substitutions

A **point mutation** changes one amino acid. This project deals exclusively with **substitutions** — one residue swapped for another — not insertions or deletions, because a substitution leaves the sequence length unchanged, which keeps the two forward passes comparable position by position.

The standard notation is A45T:

- A — the **wild-type** residue (what is normally there),
- 45 — the position, **counting from 1**,
- T — the **mutant** residue.

“Wild type” means the reference, unmutated version. It is the control condition in every experiment here.

In code, `parse_mutation("A45T")` returns a `Mutation` object. Note carefully:

```
mutation = parse_mutation("A45T")
mutation.wt_aa      # "A"
mutation.mut_aa     # "T"
mutation.one_indexed_pos # 45 - as written in the string
mutation.seq_pos     # 44 - 0-indexed, for indexing into Python strings
```

The conversion between 1-indexed biology and 0-indexed code is the single most common source of silent errors in this kind of pipeline, which is why it is done in exactly one place. Appendix B lays out every index convention in the project.

`mutation.apply(sequence)` produces the mutant sequence **and checks that the wild-type residue named in the string is actually the residue at that position**. If it is not, it raises rather than proceeding, because a mismatch means the mutation table and the sequence disagree about numbering, and every downstream number would be wrong in a way that looks fine.

What substitutions do

Roughly, in decreasing order of severity:

- Replacing a buried hydrophobic residue with a charged one destabilises the fold.
- Removing a catalytic residue kills the enzyme.
- Introducing a proline into a helix breaks the helix.
- Changing a surface residue to a similar one usually does nothing.

Alanine scanning exploits this. Alanine has a very small side chain, so mutating a residue to alanine removes its side chain while leaving the backbone intact. If function drops, the side chain mattered. This project uses alanine substitution as its functional probe in the benchmark (Part 7.5); positions that are already alanine are mutated to glycine, which is smaller still.

1.5 Measuring what a mutation does

Two data sources, with different characters.

Deep mutational scanning and ProteinGym

Deep mutational scanning (DMS) is an experimental technique: make a library containing every possible single substitution in a protein, subject the whole library to a selection for function, sequence what survives, and read off a fitness score per variant. One experiment can measure thousands of mutations.

ProteinGym is a curated benchmark collecting many published DMS assays into a uniform format — one CSV per assay, with a mutant column (A45T), a `mutated_sequence` column, a continuous `DMS_score`, and a binarised `DMS_score_bin` where 1 means functional and 0 deleterious.

The loader `load_proteingym` in `src/utils/dataloaders.py` reads these. One design decision worth flagging: it recovers the wild-type sequence by *reverting* the substitution in the `mutated_sequence` column, rather than matching the assay against an external reference FASTA. This is self-consistent by construction and cannot suffer the numbering drift that the external-reference route is prone to. Rows where the mutation string and the sequence disagree are dropped and counted.

ClinVar

ClinVar is a public database of human genetic variants annotated with clinical significance: *Pathogenic*, *Likely pathogenic*, *Benign*, *Likely benign*, or *Uncertain significance* (VUS).

It is a different kind of evidence — clinical rather than experimental, and subject to reporting bias, since variants get submitted when someone had a reason to look. The loader keeps only the clearly pathogenic and clearly benign labels and drops VUS, because the point of using ClinVar here is a clean binary contrast and VUS rows would add exactly the label noise that makes a correct circuit look unfaithful.

ClinVar is not auto-downloadable in a usable form for this project. Its variant summary gives HGVS notation (NM_007294.3:c.181T>G), and converting that into a (wt_sequence, mutation) pair requires choosing a transcript set to trust — a choice that would silently determine the results. `scripts/fetch_assets.py` therefore writes the expected schema and leaves that join to you.

Synthetic data

There is a third source: `synthetic_variants` generates proteins with a planted HExxH motif (a real zinc-binding motif; x means any residue) and labels mutations inside the motif as deleterious.

This exists because it gives the pipeline a **known ground truth**. If circuit discovery cannot recover a motif that was deliberately planted, it will not recover a real active site, and you want to learn that in thirty seconds rather than after a day of GPU time. Use `data.source=synthetic` when changing the discovery algorithm.

1.6 Where 3D structures come from

The Protein Data Bank

The **PDB** is a public archive of experimentally determined protein structures. Each entry has a four-character ID like 1A2Y or 4HHB and contains the 3D coordinates of the atoms, determined by X-ray crystallography, NMR spectroscopy, or cryo-electron microscopy.

A PDB file is a fixed-width text format, one line per atom:

```
ATOM      12  CA  LEU A  15      24.317  10.472  33.196  1.00 18.42      C
```

The columns are positional, not delimited — column 13–16 is the atom name, 18–20 the residue name, 22 the chain, 23–26 the residue number, and 31–54 the x, y and z coordinates in ångströms. Biopython parses by character offset, which is why the test fixture in `tests/conftest.py` documents the column layout in detail: a line that is “close enough” produces wrong coordinates rather than a parse error.

α -carbons

Each amino acid has a central carbon atom called the **α -carbon** ($C\alpha$), sitting between the backbone nitrogen and the backbone carbonyl carbon. It is the conventional single-point representative of a residue’s position.

This project computes all distances between C α atoms. Using C α rather than, say, the side-chain centroid is standard, is robust to side-chain flexibility, and is what the 6–8 Å contact convention is calibrated against.

What is missing from a structure, and why it matters

This is the part that trips people up, so it gets its own list.

Not every residue has coordinates. Flexible regions do not diffract coherently, so a crystallographer cannot place them. They are simply absent from the file. A 300-residue protein might have 270 resolved residues with a 30-residue hole in the middle and 8 missing from each end.

Numbering may not start at 1. PDB files use *author numbering*, which often follows the mature protein after a signal peptide has been cleaved, or follows a historically established numbering for that protein family. Residue 1 in the file may be residue 24 of the sequence you have.

Constructs contain extra material. Expression tags (a run of histidines, for example) get added to help purification and may or may not appear in the file.

There may be several chains. Entry 4HHB (haemoglobin) has four. You must pick one.

There may be several models. NMR structures contain an ensemble, typically twenty. You take one.

Non-protein content is present. Water molecules, ions, bound drugs, buffer components. These appear as HETATM records and must be filtered out — with one exception the loader makes deliberately: MSE (selenomethionine) is a methionine with selenium substituted for sulfur, used to help solve the phase problem in crystallography. It is chemically a methionine and should be kept.

The consequence of all of this: **you cannot assume that position *i* of your sequence is residue *i* of the structure.** Part 6.2 goes into why this is catastrophic if ignored and Part 6.3 into what we do instead.

Part 2 — The machine learning

This part builds up to ESM-2 from nothing. If you have trained a transformer, skim to Section 2.8, which is where the project-specific content starts.

2.1 What a neural network is

A neural network is a function. It takes numbers in and gives numbers out, and it has adjustable **parameters** (also called **weights**) that determine what function it computes.

The smallest useful piece is a **linear layer**:

$$y = x W + b$$

where x is a vector of length d_{in} , W is a matrix of shape (d_{in}, d_{out}) , b is a vector of length d_{out} , and y comes out with length d_{out} . W and b are the parameters.

Stacking linear layers achieves nothing, because a composition of linear functions is linear. So between them you put a **non-linearity** — a simple fixed function applied element by element. The most common is **ReLU**:

$$\text{ReLU}(x) = \max(0, x)$$

It passes positives through and zeroes negatives. This project uses ReLU inside the sparse autoencoder and **GELU**, a smooth relative of it, inside ESM-2.

A **layer** in the loose sense is one such transformation. A **deep** network is many of them stacked. The intermediate vectors between layers are called **activations**, and they are the object this entire project studies: not the weights, which are fixed once training ends, but the activations, which are what the network computes about a particular input.

Shapes

You will see shapes written like `[batch, seq, d_model]`. That means a three-dimensional array (a **tensor**) where:

- **batch** — how many sequences are being processed at once,
- **seq** — how many tokens are in each sequence,
- **d_model** — how many numbers represent each token.

For ESM-2 8M, `d_model` is 320. For ESM-2 35M it is 480. That number — the **width** or **hidden size** — recurs constantly.

2.2 Training: loss, gradients, backpropagation

Loss

A **loss function** measures how wrong the network's output is, as a single number. Training means adjusting the parameters to make it smaller.

For predicting a category out of many — which amino acid goes here? — the standard loss is **cross-entropy**. If the model assigns probability p to the correct answer, the loss is $-\log p$. Confident and right gives a loss near zero; confident and wrong gives a very large loss.

For reconstructing a vector — which is what the sparse autoencoder does — the standard loss is **mean squared error (MSE)**: the average of the squared differences, element by element.

Gradients

The **gradient** of the loss with respect to a parameter is a number saying: if I increase this parameter slightly, does the loss go up or down, and how fast? It is the derivative.

If you have the gradient for every parameter, you can improve the network: nudge each parameter a small step in the direction that decreases the loss. That is **gradient descent**. The size of the step is the **learning rate**.

Backpropagation

Backpropagation is the algorithm that computes all those gradients efficiently. It is the chain rule from calculus, applied systematically.

The chain rule says that if z depends on y and y depends on x , then

$$dz/dx = (dz/dy) \cdot (dy/dx)$$

A network is a long chain of such dependencies. Backpropagation starts at the loss, computes the gradient with respect to the last layer's output, then works backwards layer by layer, multiplying by each layer's local derivative. One backward pass gives you the gradient for every parameter — and, importantly for this project, for every *activation* too.

That last point matters. In Part 5.10 we compute the gradient of a metric with respect to the sparse autoencoder's internal features, in order to estimate what would happen if we changed them. That is backpropagation used for measurement rather than for training.

In PyTorch this is `loss.backward()`, and `torch.autograd.grad(value, tensor)` when you want the gradient with respect to one specific tensor without touching anything else.

Forward and backward passes

A **forward pass** is running an input through the network to get an output. A **backward pass** is propagating gradients back through it. A forward pass is cheap; a backward pass costs roughly twice as much. This ratio is why Part 5.10 exists: scoring five thousand features one at a time with forward passes is prohibitive, while one backward pass scores all of them approximately.

2.3 Optimisers and schedules

Plain gradient descent works but converges slowly. **Adam** is the standard improvement and is what this project uses. It maintains, per parameter, a running average of recent gradients (the **first moment**) and of recent squared gradients (the **second moment**), and scales each parameter's step by dividing by the square root of the second moment. The effect is that parameters with consistently small gradients take proportionally larger steps.

The **betas** — (0.9, 0.999) by default — control how long those two averages remember.

A **learning-rate schedule** changes the step size over training. The project uses **linear warmup then cosine decay**:

- *Warmup*: start at a tiny learning rate and ramp up over the first few hundred steps. Early in training the parameters are far from anything sensible and large steps can do damage that takes a long time to undo. In this codebase warmup matters more than usual, because the autoencoder's decoder bias is initialised from the data and the encoder starts as the decoder's transpose, so the first few hundred steps involve large, badly scaled updates that can push the decoder off the unit sphere faster than renormalisation recovers it.
- *Cosine decay*: smoothly reduce the learning rate afterwards, following a cosine curve down to about 10% of peak, so the run settles rather than bouncing around a minimum.

Gradient clipping caps the total size of the update, so one anomalous batch cannot throw the model. The project clips global gradient norm at 1.0.

2.4 Language models and the masked objective

A **language model** assigns probabilities to sequences. There are two families.

Autoregressive (GPT-style) models predict the next token given all previous ones. They read left to right.

Masked (BERT-style) models hide some tokens and predict them from the context on *both* sides. Training shows the model

M K T A Y [MASK] A K Q R Q

and asks what the masked position should be. The correct answer is known, so the loss is cross-entropy against it.

ESM-2 is a masked language model. This is the right choice for proteins: a protein is not read left to right by anything in nature, and a residue's identity is constrained by neighbours in both directions — and, through folding, by residues far away in either direction.

The consequence for us is that ESM-2 has a **masked-LM head** producing, at every position, a probability distribution over the 33 tokens in its vocabulary. That distribution is the readout we intervene on in Part 5.

Logits

Before the probabilities there are **logits** — raw, unnormalised scores, one per vocabulary entry. The **softmax** function turns logits into probabilities:

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

Log-probabilities are the logarithm of these, which is what you want numerically: probabilities of 10^{-15} underflow, their logarithms do not.

A fact used repeatedly in Part 5: the *difference* of two log-probabilities at the same position equals the difference of their logits, because the softmax denominator is shared and cancels.

$$\log p_a - \log p_b = z_a - z_b$$

This makes a **logit difference** invariant to anything that shifts all the logits at a position equally — which is exactly the property you want in a measurement, since a uniform shift is not a meaningful change in the model’s opinion.

2.5 Tokens, vocabularies and embeddings

A network takes numbers, not letters. **Tokenisation** converts text to integers.

For text models this is complicated. For proteins it is trivial: one amino acid, one token. ESM-2’s vocabulary has 33 entries, in a fixed order shared by every ESM-2 checkpoint from 8M to 15B parameters:

```
<cls> <pad> <eos> <unk>
L A G V S E R T I D P K Q N F Y M H W C X B U Z O . - <null_1> <mask>
```

The four **special tokens** at the start do specific jobs:

- **<cls>** — prepended to every sequence. Originally a “classification” token whose final representation summarises the whole sequence.
- **<pad>** — filler, so sequences of different lengths can sit in one batch.
- **<eos>** — appended to mark the end.
- **<unk>** — anything unrecognised.

And **<mask>** at the end is what replaces a hidden residue during training, and what we use to ask the model “what belongs here?” at inference time.

The <cls> token is why token positions are offset by one from sequence positions. Residue 1 of the protein is token 1, not token 0. Getting this wrong shifts every reported residue by one — a mistake that produces plausible output. `seq_to_token_pos` and `token_to_seq_pos` are the only sanctioned conversions in the codebase, and `token_to_seq_pos(0)` deliberately raises, because **<cls>** is not a residue.

Embeddings

An **embedding layer** is a lookup table mapping each token ID to a vector of length `d_model`. These vectors are learned. After the embedding layer, a sequence of n tokens has become a tensor of shape `[1, n, d_model]`, and everything from there on is arithmetic on real numbers.

Position information

Attention (next section) has no inherent notion of order — it would give the same answer for a shuffled sequence. Position must be injected.

ESM-2 uses **rotary position embeddings (RoPE)**, which rotate the query and key vectors by an angle proportional to position. The dot product between two rotated vectors then depends on their *relative* separation, which is the quantity that should matter. This is

worth knowing because the test suite constructs an ESM-2 config and must specify `position_embedding_type="rotary"` to match the real architecture.

ESM-2 also uses **token dropout**: during training, masked positions have their embeddings zeroed and the remaining ones rescaled. This is an ESM-specific detail, enabled by `token_dropout=True` in the config, and the test fixtures replicate it so the module graph under test is the real one.

2.6 Attention

Attention is the mechanism that lets one position in the sequence read from another.

The mechanism

For each position, the model computes three vectors by multiplying the position's current representation by three learned matrices:

- a **query** q — “what am I looking for?”
- a **key** k — “what do I offer?”
- a **value** v — “what do I hand over if selected?”

Position i compares its query with every position's key by taking a dot product. Large dot product means “this is relevant to me”. Those scores are scaled by $1/\sqrt{d_{\text{head}}}$ (to keep them from growing with dimension) and passed through a softmax to become **attention weights** that are non-negative and sum to one across positions.

The output at position i is the weighted average of all positions' value vectors, using those weights:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_{\text{head}}}) V$$

Multiple heads

A transformer does this several times in parallel with different learned matrices. Each parallel copy is a **head**. ESM-2 8M has 20 heads per layer. Different heads learn different relationships. Their outputs are concatenated and mixed by another linear layer.

The thing to remember

Attention weights sum to one. Every position always distributes exactly one unit of attention. This means attention weights always *look* like an explanation — there is always somewhere the model “looked most”. It says nothing about whether the values retrieved from there influenced the output. Part 3.3 develops this.

2.7 The transformer block

One transformer **block** (or **layer**) does two things in sequence:

1. **Attention sub-layer.** Normalise, run multi-head attention, add the result back to the input.

2. **Feed-forward sub-layer.** Normalise, run a two-layer MLP (expand to `intermediate_size`, apply a non-linearity, project back to `d_model`), add the result back.

The MLP is where most of the parameters live. For ESM-2 8M, `d_model` is 320 and `intermediate_size` is 1280, so the MLP expands fourfold and contracts back.

LayerNorm

Layer normalisation rescales a vector to have mean 0 and variance 1 across its features, then applies a learned scale and shift. It keeps activations in a consistent range so training is stable.

Where you put it matters enormously for this project:

- **Post-LN** (original 2017 transformer): $x \leftarrow \text{LayerNorm}(x + \text{sublayer}(x))$.
- **Pre-LN** (modern, and what ESM-2 uses): $x \leftarrow x + \text{sublayer}(\text{LayerNorm}(x))$.

In pre-LN, the normalisation is applied to the *input* of the sub-layer, and the sub-layer's output is added to an unnormalised running total. That running total is the residual stream, and it is why pre-LN is the architecture that makes this project's approach clean. Section 2.8 explains.

Adding it up

For ESM-2 8M: 6 blocks, each with 20 attention heads over a 320-dimensional representation and a 1280-wide MLP. About 8 million parameters. For ESM-2 35M: 12 blocks, `d_model` 480, `intermediate_size` 1920.

2.8 The residual stream

This is the central concept of the whole project. Read it twice.

What it is

Look again at the pre-LN block:

```
x ← x + attention(LayerNorm(x))
x ← x + mlp(LayerNorm(x))
```

Notice that `x` is never replaced — it is only ever **added to**. There is a vector at each position that starts as the token embedding and accumulates contributions as it passes through every block. That vector is the **residual stream**.

The shortcut `x + ...` is a **residual connection** (or skip connection), introduced to make deep networks trainable: gradients flow back through the addition unimpeded, so depth does not kill learning.

Why it is the right thing to study

The interpretability consequence is the important part. Because every sub-layer *reads* from the residual stream (via the `LayerNorm`) and *writes* to it (via the addition), the residual stream is the network's **communication channel**. Information produced at layer 2 that

layer 5 needs must be written into the residual stream and carried there. Nothing else persists.

So:

- If you want to know what the model has computed by a certain depth, read the residual stream at that depth.
- If you want to change what the rest of the model sees, write to the residual stream at that depth.

Reading and writing the same object is what makes intervention clean. If we hooked something internal to a block instead — an attention pattern, an MLP activation — we would be able to observe it but would have to reason separately about how a change there propagates. Hooking the block output means the tensor we measured is literally the tensor we overwrite.

In code

`src/models/esm_hooks.py` attaches a **forward hook** to each transformer block. A forward hook is a PyTorch callback that fires after a module computes its output, receives that output, and may return a replacement.

```
with ESMActivationExtractor(model, [3, 4]) as extractor:
    model(**batch.as_model_kwargs())
    acts = extractor.activations[3]  # [batch, seq, d_model]
```

A HuggingFace `EsmLayer` returns a tuple whose first element is the hidden states; later elements may hold attention weights. The helper `_normalise_layer_output` splits that apart and `_rebuild_layer_output` puts it back, so patching preserves whatever the caller asked for.

Two robustness notes that cost real debugging time to learn:

The layer path differs between model classes. `EsmModel.encoder.layer` exists; `EsmForMaskedLM.encoder` does not — the blocks are at `EsmForMaskedLM.esm.encoder.layer`. Published patching code routinely hard-codes one and breaks on the other. `resolve_encoder_layers` tries the known paths and, when all fail, raises an error naming the top-level children it actually found.

Hooks must always be removed. A hook stays attached until removed and fires on every subsequent forward pass in the process. A leaked hook silently corrupts every later measurement. Every hook registration in this codebase is scoped by a context manager, so even an exception mid-forward cannot leave one behind. There is a test for exactly this (`test_hooks_removed_even_when_forward_raises`).

2.9 ESM-2

ESM stands for Evolutionary Scale Modeling. ESM-2 is a family of masked language models trained by Meta on UniRef, a clustered database of roughly 250 million protein sequences. The family ranges from 8 million to 15 billion parameters.

The training signal is only sequence. No structures, no functional annotations, no labels. Just: here is a protein with some residues hidden, guess them.

What makes this interesting is what falls out. Because residues that are in physical contact co-evolve — a mutation at one is tolerated only if compensated at the other — a model

that predicts masked residues well must implicitly represent which residues are in contact. Contact is a 3D fact, and it has been learned from 1D data.

The two checkpoints used here

	8M (default)	35M (opt-in)
HuggingFace ID	facebook/esm2_t6_8M_UR50D	facebook/esm2_t12_35M_UR50D
Blocks	6	12
d_model	320	480
Attention heads	20	20
MLP width	1280	1920
Vocabulary	33	33

The 8M model is the default because the whole pipeline — autoencoder training plus a full circuit sweep — fits comfortably in 8 GB of unified memory on an M2. Switch with `model=esm2_35m` when you want final numbers; expect roughly four to five times the wall-clock cost, with the layer sweep being the expensive part.

Which layers to hook

The configs default to middle layers: [2, 3, 4] for the 8M model, [4, 6, 8] for the 35M. The reasoning is that layer 0 is still close to the raw embedding and carries mostly token identity, while the final layer has been specialised toward the output head and its representation is shaped by the prediction task rather than by general structure. The middle is where the interesting, structured features tend to live.

2.10 Scoring a mutation with ESM-2

ESM-2 does not output “this mutation is bad”. You have to construct a score.

Masked marginals

The standard approach. Take the wild-type sequence, replace the position of interest with `<mask>`, run the model, and read the log-probabilities it assigns at that position:

```
score = log p(mutant residue | masked context) - log p(wild-type residue | masked context)
```

A negative score means the model finds the mutant less plausible than the wild type, which correlates with loss of function. This is implemented as `ESMWrapper.masked_marginal_score` and is used as the functional probe in the benchmark (Part 7.5).

The metric used for patching

For path patching we need a slightly different thing, and the distinction matters enough to spell out.

If we masked the mutated position, the wild-type and mutant sequences would become **identical** — the only place they differ is exactly the position we just masked. There would be nothing to patch.

So the patching metric leaves the sequences unmasked. Both runs see their full sequence, and the readout at the mutated position is:

```
metric = log p(wild-type residue) - log p(mutant residue)
```

evaluated on whichever sequence is being run. Under the wild-type input this is high — the model sees the wild-type residue in context and is comfortable with it. Under the mutant input it is lower. The gap between those two numbers is the effect of the mutation on the model, and it is the quantity everything in Part 5 tries to decompose.

This is `LogitDiffMetric` in `src/interpretability/path_patching.py`.

The sign convention

Every metric in this project is defined so that **higher means more wild-type-like**. That single convention makes every subsequent quantity interpretable without a sign table: a patch that raises the metric moved the model back toward its wild-type behaviour, and a patch that lowers it pushed further away.

Part 3 — Interpretability: what goes wrong

3.1 What we want from an explanation

Suppose ESM-2 says a mutation is damaging. We want to know why. An explanation that would satisfy us has three properties.

It is faithful. It describes what the model actually did, not a plausible story about what it might have done. A faithful explanation makes predictions: if you say feature f is what drives the judgement, then disabling f should change the judgement.

It is mechanistic. It names parts of the computation and how they connect, not just which inputs were important. “The model is sensitive to position 45” is an input-attribution claim. “Layer 3 computes a hydrophobic-pocket feature that layer 5 reads and converts into a stability judgement” is a mechanistic claim. The second is harder to establish and much more useful.

It is checkable. There should be some way to be wrong. For most interpretability work there is not — you produce a heatmap, it looks reasonable, and there the matter rests. Proteins offer an unusual opportunity here, because we have independently measured 3D structures to check against.

The rest of this part explains why the standard methods fail the first two, and Part 4 and Part 5 build the machinery that passes them.

3.2 Post-hoc saliency

Post-hoc methods take a trained model and a prediction and produce a score per input element, usually visualised as a heatmap. Three are relevant here, and all three are implemented in `src/interpretability/baselines.py` as the comparison baselines.

Raw attention. Read the attention weights from the position of interest and call them importance. Cheap: they were computed anyway.

Gradients. Compute $\partial \text{output} / \partial \text{input}$. Large magnitude means the output is sensitive to that input, at least locally.

Integrated gradients. A refinement of plain gradients that fixes a specific failure mode. Part 7.3 derives it.

All three answer the question “what is the model’s output locally sensitive to?” None answers “what did the model use?” — and those questions come apart.

3.3 Why attention is not explanation

This deserves spelling out, because attention heatmaps are the default in the protein language model literature and they look convincing.

The structural argument

Attention weights are a convex combination — non-negative and summing to one. The output at a position is a weighted average of value vectors. So:

A weight can be large while the value it retrieves is near zero. The model attends heavily to position 45 and receives almost nothing from it. The heatmap is bright; the influence is nil.

A weight can be small while the value is large. A 2% weight on a value vector with large magnitude contributes more than a 40% weight on a small one.

Attention is one path among many. A residue can influence the output through the MLP sub-layer, or through a multi-hop route (position 45 influences position 20 at layer 2, which influences the output at layer 5), without ever receiving high direct attention from the readout position.

Attention always sums to one. Even on an input the model is completely indifferent to, the weights distribute exactly one unit. There is always a maximum. A heatmap will always show a pattern, whether or not one exists.

The empirical argument

This has been tested. There is a well-known line of work — *Attention is not Explanation*, and the partial rebuttal *Attention is not not Explanation* — finding that you can often construct substantially different attention distributions that leave the model’s prediction essentially unchanged. If several different “explanations” produce the same output, none of them is *the* explanation.

What follows

You cannot establish causal influence by observing a forward pass, however carefully. You have to change something and see what happens. That is the whole argument for Part 5.

This is not to say attention maps are useless. They are informative about the model’s information routing and they are free. They are just not evidence of what mattered, and the benchmark in Part 7 quantifies the gap rather than asserting it.

3.4 Polysemantic neurons

Suppose we accept that we must intervene. On what?

The obvious unit is the **neuron** — one dimension of the residual stream, one coordinate of the 320-dimensional vector at each position. We could patch individual neurons.

It does not work, because neurons are **polysemantic**: a single neuron responds to several unrelated things. In vision models a neuron might fire for cat faces, car fronts *and* certain

textures. In a protein model a neuron might fire for histidines, for buried positions, *and* for a particular structural motif, with no consistent relationship between them.

Patching such a neuron tells you that “the mixture of three unrelated things mattered”, which is not an explanation. You cannot build a mechanistic account out of units that do not mean one thing.

3.5 Superposition

Polysemanticity is not an accident. There is a reason for it, and understanding the reason tells you how to fix it.

The counting argument

A protein language model needs to represent a great many properties: this residue is hydrophobic; this region is a helix; this position is in a binding pocket; this protein is a kinase; this position co-evolves with position 112; and so on into the thousands.

The residual stream at each position has 320 dimensions.

If each property needed its own dimension, the model could represent at most 320 properties. It evidently represents more. So properties must share dimensions.

How sharing is possible

The key observation is **sparsity**: at any given position, only a few properties are active. A residue is not simultaneously hydrophobic, charged, in a helix, in a sheet, buried and exposed. Most features are off at most positions.

Under sparsity, a network can pack n features into d dimensions with n far larger than d by assigning each feature a direction and accepting that the directions cannot all be orthogonal. Two features that are almost never active at the same time can share nearly the same direction with little cost, because the interference between them rarely arises.

This is **superposition**. The consequence is that any single coordinate — any neuron — picks up a component from many features, which is exactly what polysemanticity looks like from the outside.

The geometric picture

There is a classical result behind this. The Johnson-Lindenstrauss lemma says that you can place exponentially many *almost*-orthogonal vectors in d dimensions, even though you can place only d exactly-orthogonal ones. “Almost orthogonal” means pairwise dot products are small but not zero — a small, tolerable amount of interference.

So a network in superposition has a dictionary of many more feature directions than it has dimensions, and reads each one out with a small error contributed by whichever other features happen to be active.

What follows

If features are directions and neurons are coordinates, then **the coordinate basis is the wrong basis**. We need to find the feature directions.

Finding them is a problem with a name: given data that is a sparse combination of unknown directions, recover the directions. That is **dictionary learning**, and Part 4 is about how to do it.

3.6 Features as directions

The assumption underlying everything that follows is the **linear representation hypothesis**: that the model represents an interpretable property as a direction in activation space, and the strength of that property as the magnitude along that direction.

Concretely, the residual stream at a position is modelled as

$$x \approx b + \sum_i z_i \cdot d_i$$

where each d_i is a unit vector (a **dictionary element**, or **feature direction**), $z_i \geq 0$ is how strongly feature i is active, b is a constant offset, and — crucially — **most of the z_i are zero**.

Some things to note about this model.

It is an assumption, not a theorem. There is good evidence for it in language models and increasing evidence in protein models, but it could be wrong or incomplete. Features that are genuinely non-linear, or represented as a manifold rather than a direction, would be missed by everything here.

Non-negativity is a modelling choice. Features are “present with some strength” or “absent”, not negatively present. This is why ReLU appears.

Sparsity is what makes it identifiable. Without the constraint that most z_i are zero, the decomposition is wildly non-unique — you could pick any basis. Sparsity is what pins it down.

The number of features exceeds the dimension. That is the whole point. In this project the dictionary is 16 times wider than the residual stream by default: 5120 features for the 320-dimensional 8M model.

Part 4 — Sparse autoencoders

4.1 Autoencoders

An **autoencoder** is a network trained to reproduce its own input. It has an **encoder** that maps the input to some intermediate representation, and a **decoder** that maps that back to the original space. The loss is the reconstruction error.

Stated like that it sounds pointless — the identity function would do. It becomes useful when you constrain the intermediate representation so that the identity is unavailable. Classically the constraint is that the intermediate is *smaller* than the input, forcing compression. This is roughly what PCA does.

A sparse autoencoder inverts that. The intermediate is *much larger* than the input — 16 times larger here — but only a few of its entries are allowed to be non-zero. The constraint is not size but sparsity.

That is precisely the structure Section 3.6 described. An overcomplete, sparsely-activated autoencoder is a dictionary learner.

4.2 Dictionary learning

The formal problem: given many observed vectors x , find a dictionary D (a set of directions) and sparse codes z such that $x \approx D z$ for each observation, with z having few non-zeros.

The dictionary is **overcomplete** when it has more elements than the space has dimensions. Overcomplete dictionaries have no unique representation for a given x in general — many combinations reproduce it — and sparsity is the criterion that selects among them.

This is a mature field (sparse coding, compressed sensing) predating its application to neural networks. What is new is the target: instead of images or audio, the observations are a transformer’s internal activations, and the recovered dictionary elements are hypothesised to be the model’s own features.

4.3 The SAE recipe

Concretely, for our case.

Input: the residual stream at a chosen layer, flattened across batch and sequence, so a matrix of shape $[n_tokens, d_in]$ with $d_in = 320$.

Encoder: subtract a learned bias, multiply by a matrix, add another bias.

```
pre_acts = (x - b_dec) @ W_enc + b_enc          # [n_tokens, d_sae]
```

Sparsify: keep only a few entries of `pre_acts`, zero the rest. How is the subject of Sections 4.4 and 4.5.

Decoder: recombine the surviving entries with their dictionary directions.

```
x_hat = latents @ W_dec + b_dec          # [n_tokens, d_in]
```

Loss: mean squared error between `x` and `x_hat`, plus an auxiliary term described in Section 4.6.

Shapes, for concreteness, with the 8M model and default config:

Tensor	Shape	Meaning
<code>x</code>	<code>[n, 320]</code>	residual stream, one row per token
<code>W_enc</code>	<code>[320, 5120]</code>	encoder
<code>b_enc</code>	<code>[5120]</code>	encoder bias
<code>latents</code>	<code>[n, 5120]</code>	sparse code, 32 non-zeros per row
<code>W_dec</code>	<code>[5120, 320]</code>	dictionary — row j is feature j's direction
<code>b_dec</code>	<code>[320]</code>	decoder bias

Note the decoder convention: `W_dec` is `[d_sae, d_in]`, so each *row* is one dictionary direction and the unit-norm constraint of Section 4.6 is over `dim=1`. Some implementations transpose this. Mixing them up is a silent bug.

Why `d_sae = 16 × d_in`

Two competing pressures. Too small and the dictionary cannot cover the model's features, so several real features get merged into one dictionary element and you are back to poly-semanticity. Too large and features get *split* — one real feature is represented by several near-duplicate dictionary elements, which fragments the analysis and wastes capacity. The factor 16 is a common working choice; `dict_mult` in `configs/sae/topk_sae.yaml` exposes it.

4.4 L1 sparsity and its problems

The traditional way to enforce sparsity is to add a penalty on the sum of absolute values of the code:

$$\text{loss} = \|x - x_{\text{hat}}\|^2 + \lambda \cdot \sum_i |z_i|$$

The L1 term pushes entries toward zero, and because of the geometry of the absolute value it pushes many of them exactly to zero rather than merely making them small. This is the same mechanism as LASSO regression.

It works, and it has two problems that matter here.

Problem one: λ has to be tuned, per layer

λ controls the trade-off between reconstruction and sparsity. There is no principled value. Worse, the right value differs by layer, because different layers have different activation scales and different intrinsic sparsity. So comparing “layer 2 versus layer 4” requires a sweep for each, and even then the two dictionaries sit at different points on their respective trade-off curves and are not directly comparable.

Since one of the project’s outputs is a table of reconstruction quality across layers, this is not a minor inconvenience.

Problem two: shrinkage

L1 does not only zero the entries it eliminates; it also shrinks the ones it keeps. If the true activation of a feature is 3.0, the L1-penalised solution might report 2.6, because the penalty applies to every non-zero entry.

The reconstruction is therefore systematically biased — every active feature is slightly too small. For our purposes this is worse than it sounds: we later patch a feature’s value from one run into another, and if both values are shrunk by an amount that depends on how many other features were active, the patch is systematically wrong in a way that varies by input.

4.5 Top-K

The alternative, and what this project uses: **do not penalise, just truncate.**

Compute all the pre-activations, keep the k largest, set everything else to zero exactly.

```
def encode(self, x):
    pre_acts = self.preactivations(x)
    values, indices = torch.topk(pre_acts, self.cfg.k, dim=-1, sorted=False)
    values = F.relu(values)
    latents = torch.zeros_like(pre_acts).scatter_(-1, indices, values)
    return latents, indices, values
```

This fixes both problems.

L0 is exactly k , by construction. L0 is the count of non-zero entries. There is no coefficient to tune, and every layer’s dictionary is at the same sparsity, so cross-layer comparison is meaningful without a sweep.

No shrinkage. The surviving values are passed through unmodified. A feature whose true strength is 3.0 is reported as 3.0.

The trade-off is explicit. Raising k improves reconstruction and worsens interpretability, monotonically and legibly. You choose the operating point directly instead of discovering it through λ .

One implementation detail

The code applies ReLU *after* the top- k selection rather than before. Selecting on pre-activations means the k slots go to the k most strongly driven directions. When at least k pre-activations are positive — which is essentially always after a few hundred training steps — this is identical to ReLU-then-top- k . It differs only in the early-training regime where fewer than k are positive, and there, clamping after selection keeps the code non-negative rather

than admitting negative activations. The docstring in `sparse_autoencoder.py` records this reasoning.

The sparse view

`forward` returns both a dense `latents` tensor of shape `[n, 5120]` and the sparse indices/values pair of shape `[n, 32]`. The dense form is convenient; the sparse form is 160 times smaller and is what you need when scanning a large corpus for a feature’s top-activating tokens. `decode_sparse` decodes directly from the sparse view without materialising the dense one, and a test asserts the two agree exactly — if they ever diverged, feature profiles and patching results would disagree in a way that is very hard to trace.

4.6 The three details that decide whether it works

A Top-K autoencoder written from the description above will train, produce plausible loss curves, and give you a mostly-dead dictionary of meaningless directions. Three additions make the difference. Each addresses a specific failure.

Detail one: unit-norm decoder, with gradient projection

The failure. Nothing in the loss prevents the model from scaling a decoder direction up by 10 and the corresponding latent down by 10. The reconstruction is identical. But now feature magnitudes are meaningless — a latent value of 0.3 for one feature and 30 for another may indicate identical strength. Worse, the top-k selection compares pre-activations across features, so arbitrary scaling actively corrupts which features get selected.

The fix, part one. Constrain every decoder row to unit L2 norm, and renormalise after every optimiser step.

```
@torch.no_grad()
def normalize_decoder(self):
    self.W_dec.data = F.normalize(self.W_dec.data, dim=1)
```

The fix, part two — the part that is usually missing. Renormalising alone fights the optimiser. The gradient of the loss with respect to a decoder row generally has a component pointing *along* that row (radial) and a component perpendicular to it (tangential). Only the tangential part can change a unit vector’s direction; the radial part only tries to change its length, and renormalisation immediately discards that change.

But Adam does not know it was discarded. It accumulates the radial component into its second-moment estimate, which inflates the denominator and shrinks the step size for that parameter. The effective learning rate drifts over training in a way that depends on how much radial gradient there happened to be.

So we remove the radial component *before* the optimiser sees it:

```
@torch.no_grad()
def remove_parallel_gradient(self):
    w = self.W_dec.data
    g = self.W_dec.grad
    radial = (g * w).sum(dim=1, keepdim=True) * w
    self.W_dec.grad -= radial
```

The correct order in the training loop is: backward, project, clip, step, renormalise. `test_remove_parallel_gradient_leaves_only_tangential_component` asserts that after projection the gradient is orthogonal to every decoder row.

Detail two: dead latents and the AuxK loss

The failure. A latent that never enters the top-k receives no gradient through the main loss. No gradient means no change; no change means it will not enter the top-k next time either. It is dead, permanently, and the dictionary has lost that much capacity.

This is not a rare edge case. Without intervention, a Top-K autoencoder typically finishes training with 50–80% of its dictionary dead. You asked for 5120 features and got 1500, and you cannot tell from the loss curve.

The fix. Track how long each latent has gone without firing. Latents past a threshold (`dead_after_tokens`, default 1,000,000) are marked dead. Then add an **auxiliary loss**: take the top `aux_k` dead latents and ask them to reconstruct the *residual error* of the main reconstruction.

```
def _auxk_loss(self, x, err):
    dead = self.dead_mask()
    pre_acts = self.preactivations(x)
    masked = pre_acts.masked_fill(~dead.unsqueeze(0), float("-inf"))
    values, indices = torch.topk(masked, aux_k, dim=-1, sorted=False)
    values = F.relu(values)
    err_hat = torch.einsum("nk,nkd->nd", values, self.W_dec[indices])
    return (err.detach() - err_hat).pow(2).sum(dim=-1).mean()
```

Two things to notice. The `masked_fill` with $-\infty$ restricts the top-k to dead latents only. And the decode omits `b_dec`, because we are reconstructing a residual, not a signal.

The effect: dead latents always receive gradient, pointed at whatever the live dictionary is failing to capture — which is exactly where a new feature would be useful. They revive, and revive into something worth having.

`aux_alpha` (default 1/32) weights this term. `aux_k = 0` disables it, which is useful only for demonstrating what happens without it.

Detail three: pre-centering and data-initialised `b_dec`

The failure. Transformer residual streams carry a large constant offset — a mean vector far from the origin. An autoencoder that does not account for it spends its early training, and some of its capacity, learning to reproduce a constant.

The fix. Subtract `b_dec` before encoding (`center_input: true`), so the encoder sees roughly zero-mean input and `b_dec` absorbs the offset. Then initialise `b_dec` to the mean of the first batch of real activations rather than to zeros:

```
@torch.no_grad()
def init_b_dec_from_data(self, x):
    self.b_dec.data = x.mean(dim=0).to(self.b_dec.dtype)
```

The geometric median is the theoretically better centre, but on residual streams the two are close and the mean costs one pass instead of an iterative solve.

A related initialisation choice: W_{enc} starts as the transpose of W_{dec} . Each encoder row begins as a matched detector for its own dictionary direction, which gives useful gradients from the first step instead of a period of random flailing.

4.7 Activation normalisation

ESM-2’s residual stream norms grow substantially with depth — a layer-8 activation is much larger in magnitude than a layer-2 one. Two consequences:

- A learning rate tuned on one layer is wrong on another.
- Reconstruction MSE is not comparable across layers, because it scales with the square of the activation magnitude.

ActivationNormalizer fits a single scalar so that activations have unit mean L2 norm, applies it before the autoencoder sees anything, and — this is the part that matters for correctness later — **stores the scale on the autoencoder’s config**, so inference reproduces training exactly.

```
norm = ActivationNormalizer.fit(activations)    # scale = 1 / mean_norm
sae_cfg.activation_scale = norm.scale
```

CausalPatcher._scale and _unscale apply and undo it around every intervention. If those were asymmetric, patching would inject a scale error into every measurement — which is one of the things test_patching_clean_into_clean_is_identity exists to catch.

4.8 Measuring a dictionary

Five numbers, computed by evaluate_reconstruction in src/validation/monosemanticity_metrics.py. They appear as Table 1 of the paper.

FVU — fraction of variance unexplained. The reconstruction error divided by the total variance of the data.

$$\text{FVU} = \frac{\sum \|x - \hat{x}\|^2}{\sum \|x - \text{mean}(x)\|^2}$$

0 is perfect; 1 means the reconstruction is no better than predicting the mean. This is the scale-free version of MSE and the one to report, because raw MSE depends entirely on the activation scale and could be pushed below any threshold by rescaling the inputs.

Explained variance is just $1 - \text{FVU}$.

L0. The average number of non-zero latents per token. For a Top-K autoencoder this is k by construction, so it functions as an assertion rather than a measurement.

Dead fraction. The proportion of the dictionary that has not fired recently. Should be near zero if AuxK is doing its job.

Cosine similarity. The average cosine between x and $\hat{x}$. Complements FVU by being insensitive to magnitude: a reconstruction can point in exactly the right direction and be systematically too short, and cosine catches that separately.

Cross-entropy recovered. The important one, and the one most often omitted.

The problem with FVU is that MSE weights every direction of the residual stream equally, and the model does not. There are a few very high-variance directions that dominate MSE, and an autoencoder can spend its entire capacity on them while discarding low-variance

directions that the model’s downstream computation actually reads. Such a dictionary has excellent FVU and destroys the model’s behaviour.

The fix is to measure behaviour directly. Run the model three ways: normally; with the layer’s activation replaced by the autoencoder’s reconstruction; and with it replaced by its batch mean (a crude ablation standing in for “this layer contributes nothing”). Then:

$$\text{CE_recovered} = (\text{CE_ablated} - \text{CE_sae}) / (\text{CE_ablated} - \text{CE_clean})$$

1.0 means the reconstruction is behaviourally free. 0.0 means it is as damaging as deleting the layer’s contribution. **If you report one number about a dictionary, report this one.**

4.9 Measuring a single feature

Dictionary-level metrics say the decomposition is good. They say nothing about whether any individual feature means something.

`profile_features` scans a corpus, records each feature’s top-activating tokens, and reports:

Firing rate — fraction of tokens where the feature is non-zero. A feature firing on 60% of tokens is not a feature, it is a bias term. One firing on 0.001% is noise or a memorised artefact.

Purity — among the top-activating tokens, the fraction carrying the single most common label (by default, amino acid identity). A feature whose top tokens are 90% histidine has purity 0.9 along the residue-identity axis.

Normalised entropy — Shannon entropy of the label distribution, divided by the maximum for that alphabet size, so it lands in [0, 1]. 0 means every top token carries the same label; 1 means uniform.

Both are reported because they fail differently. A feature split evenly between exactly two residues — say phenylalanine and tryptophan, which would be a sensible “large aromatic” feature — has purity 0.5, which looks mediocre, but entropy near 0.23 over a 20-letter alphabet, which correctly says it is highly concentrated. Purity alone would call it noise; entropy alone would miss how concentrated it is.

The caveat that matters

Low purity over amino acids is evidence of almost nothing.

The features we care about are not residue-identity detectors. A feature that fires on “buried hydrophobic position in a β -sheet” will have low amino-acid purity because several different residues satisfy that description. That is the feature being interesting, not the feature being bad.

So the summary statistic `frac_purity_above_0.5` is reported as a **diagnostic, not a target**. A dictionary consisting entirely of residue-identity detectors has learned the input alphabet, not the model’s computation. You want *some* high-purity features (they confirm the method finds real things) and plenty of low-purity ones (they are where the interesting structure lives). The way to characterise a low-purity feature is to label its top tokens by something other than identity — secondary structure from DSSP, burial, distance to the nearest ligand — which is left as an extension.

Part 5 — Causality and path patching

This is the heart of the project. Parts 1–4 built the vocabulary; this part is the method.

5.1 Correlation and causation inside a network

The everyday version of the distinction: ice-cream sales correlate with drowning deaths. Neither causes the other; summer causes both. To establish causation you intervene — change one thing while holding others fixed — and see whether the outcome moves.

Inside a neural network the same distinction applies, and interestingly, the usual obstacle to intervention does not. In medicine you cannot randomise people into smoking. In a neural network you can set any internal value to anything you like, as many times as you like, deterministically and for free. **We have perfect experimental control over the system we are studying.** Almost nothing else in science offers that.

Failing to use it — settling for observational heatmaps — is the thing that makes most interpretability work weaker than it needs to be.

5.2 Interventions and the do-operator

Judea Pearl’s causal framework distinguishes:

- $P(Y \mid X = x)$ — “the distribution of Y among cases where X happened to be x .” Observational.
- $P(Y \mid \text{do}(X = x))$ — “the distribution of Y if we *set* X to x .” Interventional.

These differ whenever something else influences both X and Y . The do operator means: reach in, fix the value, sever whatever normally determines it, and let everything downstream proceed as usual.

In a transformer, do is a forward hook. We run the forward pass, and when execution reaches layer L , we overwrite part of the residual stream and let the remaining layers compute on the modified value. Everything upstream is unchanged; everything downstream sees the intervention.

5.3 Activation patching

The standard experimental design, sometimes called **activation patching**, **causal tracing** or **interchange intervention**.

You need two inputs that differ in one respect:

- a **clean** input — here, the wild-type sequence;
- a **corrupted** input — here, the point mutant.

And a **metric** that reads out the behaviour of interest (Section 5.4).

Then the procedure:

1. Run the clean input. Record the metric. Call it m_{clean} .
2. Run the corrupted input. Record the metric. Call it m_{corrupt} .
3. Run the corrupted input again, but at layer L replace some component with the value it had in the clean run. Record the metric. Call it m_{patched} .

If m_{patched} moves back toward m_{clean} , the component you patched carries information the model uses. If $m_{\text{patched}} \approx m_{\text{corrupt}}$, it does not.

Denoising and noising

The direction above — restoring clean values into a corrupted run — is **denoising**. It asks *“is this component sufficient to restore the behaviour?”*

The reverse — inserting corrupted values into a clean run — is **noising**, and asks *“is this component necessary?”*

They are not equivalent, and redundancy is where they come apart. If two components each independently carry the needed information, restoring either one alone recovers the behaviour (both look sufficient), while removing either one alone changes nothing (neither looks necessary).

This project does **denoising**, and the greedy selection of Section 5.12 is partly a response to exactly the redundancy that denoising is prone to over-reporting.

Why a point mutation is an unusually good corruption

In language-model interpretability, choosing the corrupted input is delicate. You typically swap a token for a random one and hope the change is localised.

Here the corruption is *given by the science*. A point substitution is:

- **minimal** — one token differs, and the test `test_setup_places_the_metric_at_the_mutated_token` asserts exactly that;
- **length-preserving** — so every other position lines up between the two runs, which is what makes position-by-position patching meaningful;
- **meaningful** — it is the intervention biologists actually perform in the lab, so the thing we are decomposing is a thing someone cares about.

5.4 Choosing the metric

The metric collapses the model’s entire output into one number that we can watch move. The choice determines what the whole analysis is about.

The default: logit difference at the mutated site

```
@dataclass
class LogitDiffMetric(Metric):
    def __call__(self, logits):
        log_probs = torch.log_softmax(logits[self.batch_index, self.token_pos], dim=-1)
        return log_probs[self.wt_token_id] - log_probs[self.mut_token_id]
```

In words: at the mutated position, how much more plausible does the model find the wild-type residue than the mutant one?

Under the wild-type input this is high. Under the mutant input the model has been shown the mutant residue in context and its opinion shifts; the value drops. That drop is the effect we are decomposing.

Three reasons this is the right default:

It is a difference, so it is shift-invariant. As noted in Section 2.4, subtracting two log-probabilities at the same position cancels the softmax denominator. An intervention that raises the model’s confidence uniformly at that position does not register — correctly, because that is not a change of opinion about *this* substitution. There is a test: `test_logit_diff_metric_is_shift_invariant`.

It is local. It reads one position, so the quantity has a clear interpretation.

It is the established readout. Logit difference is the standard metric in the activation-patching literature, and using it keeps results comparable.

Why the mutated position is not masked

This is worth being explicit about, because it is a natural thing to try and it does not work.

The usual ESM-2 variant score masks the position of interest. But if we masked the mutated position, the wild-type and mutant sequences would become **identical** — the only position where they differ is the one we just replaced with <mask>. There would be no corruption to patch.

So the patching metric leaves both sequences intact. The masked version still has a role: `ESMWrapper.masked_marginal_score` uses it as an independent functional probe in the benchmark (Section 7.5), where we are asking a different question about a different position.

The alternative metric

`SequenceLogLikelihoodMetric` reads the mean log-likelihood the model assigns to all residues actually present, rather than one position. Coarser, and useful as a robustness check: a circuit that moves the local metric but never shifts the model’s overall view of the sequence supports a narrower claim than one that moves both.

5.5 Direct causal effect

With the metric fixed, the quantities are simple.

Direct causal effect:

$DCE = m_{\text{patched}} - m_{\text{corrupt}}$

How far the patch moved the metric. Positive means toward wild-type behaviour, by the sign convention of Section 2.10.

Normalised DCE:

$$\text{normalized_DCE} = (m_{\text{patched}} - m_{\text{corrupt}}) / (m_{\text{clean}} - m_{\text{corrupt}})$$

The same quantity as a fraction of the total gap the mutation opened. This is the interpretable one:

- **1.0** — this component alone explains the entire effect of the mutation.
- **0.5** — it explains half.
- **0.0** — it explains none.
- **Negative** — patching it moved the model *further* from wild-type behaviour, which happens and is informative.

The denominator problem

If the mutation barely moved the metric, the denominator is near zero and the ratio explodes. A component with a trivial absolute effect reports a normalised effect of 400%.

The code returns 0.0 when $|\text{gap}| < 1\text{e-}8$ rather than dividing, and `discover_circuits.py` prints a warning when the gap is below $1\text{e-}3$, since normalised values in that regime are unstable even if finite. The benchmark script skips such variants entirely — including them would let every method score identically on noise, which flatters the weaker methods.

`test_normalized_dce_is_zero_when_there_is_no_gap` pins this behaviour.

5.6 Patching features, not neurons

Everything so far would work on raw neurons. Part 3.4 explained why that gives uninterpretable answers. So the patch operates in the autoencoder’s basis.

The procedure at layer L :

1. Run clean, capture the residual stream `a_clean`, encode to `z_clean`.
2. Run corrupted, capture `a_corrupt`, encode to `z_corrupt`.
3. Build `z_patched`: a copy of `z_corrupt` with feature f (at chosen positions) replaced by the corresponding entries of `z_clean`.
4. Turn `z_patched` back into a residual-stream vector and continue the forward pass.

Step 4 is where it gets interesting.

Which positions

`positions=None` patches the feature at every sequence position, answering “does this feature matter anywhere?”. Passing a single position answers the sharper question of whether it matters *there*.

`positionwise_effect` runs the second form at every position in turn and returns a vector of per-position effects. **This is what turns a feature into a set of residues** — a feature can matter greatly in aggregate while its effect is concentrated at three sites, and those three sites are what Part 6 checks against the structure.

Pruning

With $k = 32$ and a 5120-element dictionary, only 32 features fire per token. A feature inactive in *both* runs has identical (zero) values in both, so patching it is provably a no-op. `active_features` returns the union of features firing in either run, which typically cuts the candidate set by roughly fifty times before any expensive computation happens.

This pruning is **exact**, not heuristic — there is no chance of discarding something that mattered. `test_active_feature_pruning_is_sound` verifies the membership test directly, and `test_inactive_features_have_zero_effect` verifies the consequence.

5.7 Error-preserving splicing ▲

This is the single most important correction in the project. If you read one technical section, read this one.

The obvious implementation, and why it is wrong

Step 4 above says “turn `z_patched` back into a residual-stream vector”. The obvious way:

```
a_new = decode(z_patched)
```

Replace the layer’s activation with the autoencoder’s reconstruction of the edited code. This is what the natural reading of the method suggests and what a lot of code does.

It is wrong. Here is the algebra.

The autoencoder does not reconstruct perfectly. Write the reconstruction error of the corrupted run as

```
 $\epsilon = a_{\text{corrupt}} - \text{decode}(z_{\text{corrupt}})$ 
```

Now compare what we want to what we get. What we *want* is the effect of changing feature f :

```
intended change = decode(z_patched) - decode(z_corrupt)
```

What the naive substitution actually does to the activation:

```
actual change = decode(z_patched) - a_corrupt
               = decode(z_patched) - decode(z_corrupt) -  $\epsilon$ 
               = intended change -  $\epsilon$ 
```

The reconstruction error ϵ has contaminated the measurement. And it is not a small contamination: ϵ is the error of reconstructing a 320-dimensional vector, while the intended change is the contribution of *one* feature out of 32 active ones. **In the typical case ϵ is larger than the signal.** Every number you report is then mostly a measurement of how badly your autoencoder fits.

The fix

Add only the difference:

```
a_new = a_corrupt + (decode(z_patched) - decode(z_corrupt))
```

Now:

```
actual change = a_new - a_corrupt
```

```

    = decode(z_patched) - decode(z_corrupt)
    = intended change

```

exactly. The error term appears in both decodes and cancels identically. What remains is attributable to feature f alone.

In code:

```

recon_patched = self.sae.decode(z_patched.reshape(-1, z.shape[-1])).reshape(b, s, d)
recon_original = out.recon.reshape(b, s, d)
return self._unscale(scaled + (recon_patched - recon_original))

```

The test that proves it

The consequence is sharp and testable: **patching zero features must be a perfect no-op.**

Under error-preserving splicing, $z_patched == z_corrupt$, so the two decodes are identical and their difference is exactly zero. The activation is untouched. The metric does not move at all.

Under naive replacement, patching zero features still substitutes $decode(z_corrupt)$ for $a_corrupt$, which differs by ϵ . The metric moves.

```

def test_error_preserving_splice_cancels_reconstruction_error(patcher, setup):
    empty = patcher.direct_causal_effect(setup, [], cache=cache)
    assert empty.dce == pytest.approx(0.0, abs=1e-5)

def test_naive_splice_does_not_cancel_reconstruction_error(wrapper, sae, setup):
    naive = CausalPatcher(wrapper, sae, layer_idx=1, splice_mode="replace")
    empty = naive.direct_causal_effect(setup, [], cache=cache)
    assert abs(empty.dce) > 1e-4

```

Both modes are implemented. `--splice-mode replace` reproduces the naive behaviour, which is worth running once so you can see the size of the gap in your own setting.

The stronger identity test

There is a second, more demanding check: `test_patching_clean_into_clean_is_identity`. Patch *every* feature of the clean run into the clean run itself. Nothing should change, because we are overwriting values with themselves.

This one test fails if the splice algebra is wrong, if activation scaling is applied asymmetrically between `_scale` and `_unscale`, or if the hook writes to the wrong element of the layer's output tuple — three independent bugs that all otherwise produce believable numbers.

5.8 The hook-leak bug

A shorter section about a bug that is easy to write and impossible to notice.

A PyTorch forward hook, once registered, fires on **every** forward pass through that module until it is removed. So this is wrong:

```

hook_handle = model.encoder.layer[layer_idx].register_forward_hook(intervention_hook)

with torch.no_grad():

```

```

patched_logits = model(corrupted_tokens).logits
baseline_corrupted_logits = model(corrupted_tokens).logits # ← also patched

hook_handle.remove()
dce = (patched_logits - baseline_corrupted_logits).abs().max().item()

```

Both forward passes run with the hook attached. The “baseline” is a patched run. The two are identical and the DCE is exactly zero, for every feature, always.

The symptom is a table of zeros — which reads as “no feature has any effect”, a scientific conclusion rather than a bug.

The fix is structural rather than a matter of care: `CausalPatcher.baselines()` computes both baselines and caches them **before any hook is registered**, and every hook in the codebase is scoped by a context manager (`CausalPatcher._patched`) so it is removed on the way out even if the forward pass raises. `test_baseline_is_measured_without_hooks` and `test_hooks_removed_even_when_forward_raises` guard both halves.

5.9 Ablation

Patching asks “does restoring this help?”. **Ablation** asks the complementary question: “does removing this hurt?”. It is what the faithfulness benchmark needs.

Two flavours:

Zero ablation sets the feature to zero. Simple, and it has a real problem: zero is not a value the activation ever naturally takes. The resulting vector is off the data manifold, and some of the damage you measure is the model reacting to an impossible input rather than to the missing feature.

Mean ablation replaces the feature with its average value over a corpus. The activation stays in a plausible region, and the measured damage is closer to “this feature’s *information* was removed” rather than “this input is nonsensical”.

Mean ablation is the better control. Zero is the default because it needs no corpus statistics; `ablate_features(..., mode="mean", mean_latents=...)` does the other. Both go through the same error-preserving splice.

5.10 Attribution patching ▲

The cost problem

Exact patching costs one forward pass per feature. With 5120 features, 3 layers and 25 variants, that is 384,000 forward passes for one benchmark run. On an M2, this does not finish.

Pruning to active features (Section 5.6) helps enormously — perhaps 100–300 candidates per layer instead of 5120 — but positional localisation multiplies the cost again by sequence length.

The linear approximation

Attribution patching estimates every feature’s effect from a single backward pass.

Let m be the metric and z the vector of latents at layer L in the corrupted run. A first-order Taylor expansion of m around z_{corrupt} :

$$m(z) \approx m(z_{\text{corrupt}}) + (\partial m / \partial z)^T (z - z_{\text{corrupt}})$$

Setting z to the patched code, which differs from z_{corrupt} only in feature f :

$$\text{DCE}_f \approx (z_{\text{clean},f} - z_{\text{corrupt},f}) \cdot (\partial m / \partial z_f)$$

The gradient $\partial m / \partial z$ for *all* features comes from one backward pass. So the whole dictionary is scored at once.

The implementation

The delicate part is making the gradient correspond to the intervention we would actually perform, rather than to some other perturbation. So the forward hook splices with the *same* error-preserving rule, with z marked as a leaf tensor requiring grad:

```
def transform(hidden):
    scaled = patcher._scale(hidden)
    out = sae(scaled.reshape(-1, d))
    z = out.latents.reshape(b, s, -1).detach().requires_grad_(True)
    holder["z"] = z
    recon = sae.decode(z.reshape(-1, z.shape[-1])).reshape(b, s, d)
    recon_ref = out.recon.reshape(b, s, d).detach()
    return patcher._unscale(scaled.detach() + (recon - recon_ref))

logits = model(**setup.corrupted.as_model_kwargs()).logits
value = setup.metric(logits)
(grad,) = torch.autograd.grad(value, holder["z"])

delta = (clean_latents - holder["z"].detach())
contrib = delta * grad
```

Note `scaled.detach()` and `recon_ref` being detached: the gradient should flow only through z , not through the path that produced the original activation.

Where it fails, and what follows

It is a first-order approximation and it inherits the standard failure of first order methods: **it is unreliable exactly where the response saturates**. If the model's output is already at a plateau with respect to a feature, the gradient is near zero, and attribution reports no effect — while a large enough change to that feature would move the output substantially.

Saturating regimes are common in trained networks, and they are often exactly where interesting circuit behaviour lives. So:

Attribution patching is used for ranking only, never for reporting.

The pipeline uses it to shortlist candidates, then re-scores the shortlist with exact path patching, and every number that appears in an output file comes from the exact stage. The docstring says so; the test suite only asserts *sign* agreement between approximate and exact, on the strongest candidates, with a tolerant threshold — because sign agreement on the top of the ranking is all the method is being asked to provide.

There is a residual risk worth stating plainly: a feature whose effect is invisible to the gradient can be dropped before exact patching ever sees it. `top_k_attribution` (default 64) controls how much slack there is. This is listed as a known limitation in Part 11.

5.11 From features to circuits

A single feature with a large effect is a finding. A **circuit** is a structured set of them.

Nodes

A `CircuitNode` is one feature at one layer, with:

- `dce` and `normalized_dce` — its individual effect;
- `token_positions` — the positions where its effect is concentrated;
- `position_effects` — the per-position DCE values;
- `residues` — the same positions as 1-indexed residue numbers.

Positions are selected by taking each feature’s positional effect profile and keeping positions whose effect is at least `position_threshold` (default 0.1) of that feature’s largest positional effect. Position 0 is dropped explicitly — it is `<cls>`, not a residue, and emitting it would produce a residue number that does not exist. `test_circuit_residues_exclude_the_cls_token` checks this.

Edges

A `CircuitEdge` is a measured influence from an upstream node to a downstream one. The procedure: patch the upstream feature in the corrupted run, then read the *downstream* autoencoder’s latents under that intervention, and compare them to the unpatched corrupted run, normalised by the clean-versus-corrupted difference for that downstream feature.

$$\text{weight} = (z_dst_patched - z_dst_corrupt) / |z_dst_clean - z_dst_corrupt|$$

This is a direct measurement of how one feature changes another, not a correlation between their activations.

Only forward edges are considered — lower layer to higher layer. The residual stream carries information in one direction, so a “backward edge” would be an artefact of the measurement, not a fact about the model.

Cost

Edge discovery is a forward pass per ordered node pair. `--no-edges` skips it, which is the right call when you only need the residues for the geometry test.

5.12 Greedy selection

The redundancy problem

Rank features by individual DCE and take the top five. This is the obvious thing and it gives you a bad circuit.

Autoencoder features are frequently **redundant**: several dictionary elements encode closely related things, and restoring any one of them recovers most of the effect on its own. Your “top five” is then five copies of one mechanism. It looks like a circuit with five components; it is a leaderboard of near-duplicates.

The fix

Grow the set greedily by **joint** effect. At each step, consider adding each remaining candidate to the set already chosen, measure the joint normalised recovery of the enlarged set, and keep the candidate that improves it most. Stop when the best available improvement falls below `min_gain` (default 0.01).

```
for f in remaining:
    res = patcher.direct_causal_effect(setup, chosen + [f], cache=cache)
    recovery = res.normalized_dce
    if recovery > best_recovery:
        best_feature, best_recovery = f, recovery
if best_feature is None or (best_recovery - current) < min_gain:
    break
```

A redundant second copy of an already-chosen feature adds nearly nothing to the joint recovery, so it is not chosen. The set stops growing when the mechanism is covered, which makes the circuit **size adaptive** rather than a fixed n .

`test_greedy_selection_is_monotonic` asserts the recovery trajectory never decreases.

Joint recovery is the headline number

The number to report for a circuit is the **joint** recovery: patch every selected feature at every selected layer **together, in a single forward pass**, and measure how much of the gap comes back.

```
with contextlib.ExitStack() as stack:
    for layer, features in selected_by_layer.items():
        stack.enter_context(patcher._patched(cache["clean_latents"], features, None))
    logits = model(**setup.corrupted.as_model_kwargs()).logits
    return float(setup.metric(logits))
```

All the hooks are registered simultaneously, so the layers interact exactly as they would in the real computation. This is deliberate and it matters — measuring each layer separately and summing would assume an additivity the model does not have.

A circuit whose parts each score well but whose joint recovery is low is not a circuit. That is the sanity check the number provides.

5.13 What conservation means

The project plan called for a test named `test_causal_conservation` verifying that “direct causal intervention sum equals total logit effect”.

That statement is false and cannot be tested, because it is not true of the model. Feature effects do not sum to the total effect. The model is non-linear: features pass through attention softmaxes, GELU non-linearities and LayerNorms, all of which make the effect of changing two things together different from the sum of changing each alone. Any test asserting additivity would be asserting something the architecture rules out.

What is exactly true, and is what the test in this repository checks:

Replacing the entire residual stream at layer L with its clean value makes everything downstream of L identical to the clean forward pass.

This is true by construction — the layers after L are a deterministic function of the residual stream at L — and it is therefore a genuine correctness check on the plumbing. If the hook writes to the wrong tensor, or the patch is applied at the wrong point in the block, this test fails.

```
def test_causal_conservation_full_patch_reaches_clean_behaviour(wrapper, wt_sequence):
    clean_acts = wrapper.residual_stream(setup.clean, [layer_idx])[layer_idx]
    clean_metric = ↔
    float(setup.metric(wrapper.model(**setup.clean.as_model_kwargs()).logits))

    handle = layers[layer_idx].register_forward_hook(
        make_patch_hook(lambda _hidden: clean_acts)
    )
    patched_metric = ↔
    float(setup.metric(wrapper.model(**setup.corrupted.as_model_kwargs()).logits))
    handle.remove()

    assert patched_metric == pytest.approx(clean_metric, abs=1e-4)
```

The general lesson: when a plan specifies a property, check whether the property is actually true of the system before writing a test that asserts it. A test of a false property either fails forever or, worse, passes because of a bug that cancels the error.

Part 6 — Grounding circuits in structure

6.1 The question

Part 5 produces a set of residues the model demonstrably relies on. That establishes something about the *model*. It does not yet establish anything about *proteins*.

Two hypotheses remain open:

Hypothesis A — the model learned biophysics. The residues it relies on form a real physical feature: an active site, a binding pocket, a folding core. They are close together in three-dimensional space even though they may be far apart in the sequence.

Hypothesis B — the model learned a shortcut. The residues are wherever the training distribution happened to put a useful statistical regularity. They have no spatial relationship.

These make different predictions about a measurement the model never saw: the experimentally determined 3D structure. That is what makes this a real test rather than an assessment of plausibility.

This is the part of the project that most interpretability work cannot do. In a text model, if you claim a circuit implements “indirect object identification”, there is no independent measurement to check against. In structural biology, there is one, and it was made by crystallographers who had never heard of ESM-2.

6.2 Why sequence position $\neq$ structure residue

The tempting shortcut: circuit residue 45 is residue 45 in the PDB file, so index into the coordinate array at 45 and compute distances.

This is almost never correct, for the reasons laid out in Section 1.6:

- Disordered termini and loops have no coordinates and are simply absent.
- Author numbering may follow a mature protein while your sequence follows the precursor.
- Expression tags add residues that may or may not appear.
- The construct used for crystallography is often a fragment.

The effect of ignoring this is a **constant offset applied to every residue**. And here is why that is so dangerous: an offset by k positions does not produce an error. It produces a perfectly well-formed set of residues, a perfectly computable mean distance, and a p-value. The output looks exactly like a real result. It is noise.

There is a test for precisely this scenario. `mini_structure` has 40 residues; the test prepends a 10-residue tag to the query sequence and asserts that the alignment shifts the mapping by exactly 10:

```
def test_alignment_recovers_an_n_terminal_offset(mini_structure):
    prefix = "GGGGGGGGGG"
    query = prefix + mini_structure.sequence
    alignment = align_sequence_to_structure(query, mini_structure)
    for struct_idx in range(mini_structure.n_residues):
        assert alignment.seq_to_struct[struct_idx + len(prefix)] == struct_idx
    assert all(i not in alignment.seq_to_struct for i in range(len(prefix)))
```

6.3 Sequence alignment ▲

The problem

Given the model's input sequence and the sequence of residues that actually have coordinates, produce a mapping from positions in the first to positions in the second, allowing for insertions and deletions.

This is **sequence alignment**, one of the oldest problems in computational biology, solved exactly by dynamic programming.

Scoring

An alignment is scored by summing a score for every aligned pair, minus penalties for gaps.

Substitution matrices give the pair scores. **BLOSUM62** is the standard: it was built by counting how often each amino acid replaces each other in blocks of related proteins that are at most 62% identical. Chemically similar substitutions score positively (leucine for isoleucine, +2), dissimilar ones negatively (tryptophan for proline, -4). Identical residues score highest.

Using BLOSUM62 rather than exact-match scoring matters because the sequence you have and the sequence in the crystal are often not identical — point differences between species, strains or engineered constructs are routine — and a scoring scheme that treats a conservative substitution as no better than a random one will misplace the alignment around it.

Gap penalties

A **gap** is a run of positions in one sequence with nothing opposite them.

Affine gap penalties charge a large cost to *open* a gap and a small cost to *extend* it:

```
aligner.open_gap_score = -11.0
aligner.extend_gap_score = -1.0
```

This encodes a real belief about the data. The dominant difference between a UniProt sequence and a crystal construct is a small number of *long* deletions — one disordered 30-residue loop — not thirty scattered single-residue deletions. With affine penalties, one 30-residue gap costs $11 + 29 = 40$, while thirty separate gaps cost $30 \times 11 = 330$. The alignment is therefore pushed toward the biologically correct interpretation.

Global alignment with free end gaps

Global alignment (Needleman-Wunsch) aligns the sequences end to end. **Local** alignment (Smith-Waterman) finds the best-matching subregion.

Global is correct here, because we want a mapping for the whole sequence, not the best-matching fragment.

But global alignment normally penalises gaps at the ends — and unresolved termini are exactly end gaps, present in nearly every crystal structure and carrying no information. So end gaps are made free:

```
aligner.end_gap_score = 0.0
```

(This single setter is used rather than the individual `target_end_gap_score`/`query_end_gap_score` attributes, because those were renamed in Biopython 1.85 and the setter works across versions.)

The output

StructureAlignment carries:

- `seq_to_struct` — the mapping, 0-indexed sequence position to 0-indexed row of the coordinate array. **Positions with no structural counterpart are simply absent**, and callers must handle that rather than assume it away.
- `identity` — fraction of aligned pairs that are identical residues.
- `coverage` — fraction of the query that has coordinates.

The sanity check

If identity comes out below `min_identity` (default 0.8), the two sequences are probably not the same protein — usually a wrong PDB ID or a wrong chain. With `strict=True` this raises; otherwise it warns and returns the mapping for inspection. Silently proceeding would produce a confident, meaningless answer.

`residues_to_indices` returns both the mapped indices **and** the list of unmapped residues, so the caller can report “3 of 7 circuit residues had no coordinates” rather than silently analysing a smaller set than it believes.

6.4 Geometry

With the mapping in hand, the geometry is elementary.

Distance matrix. All pairwise Euclidean distances between C α atoms, in ångströms.

```
def ca_distance_matrix(coords):
    diff = coords[:, None, :] - coords[None, :, :]
    return np.sqrt((diff**2).sum(axis=-1))
```

Mean pairwise distance. The average over all unordered pairs. This is the `d_circuit` of the project plan:

$$\bar{d} = (2 / (K(K-1))) \cdot \sum_{\{i < j\}} \|r_i - r_j\|$$

Radius of gyration. The root-mean-square distance of the residues from their own centroid. Less sensitive than the mean pairwise distance to a single outlying residue, so re-

porting both tells you whether a circuit is genuinely compact or is a tight core plus one straggler.

One small decision with real consequences

For fewer than two residues, both functions return **NaN**, not 0.0.

A single residue is not “maximally compact” — it is *unmeasured*. Returning 0.0 would sail through any threshold check and report a perfect result from no data. This is the kind of default that produces a table of spurious successes. `test_mean_pairwise_distance_is_nan_for_a_single_residue` pins it, and `SpatialClusteringResult.verdict` reports “undetermined” in that case.

6.5 Permutation tests ▲

Why the 6 Å threshold is not enough

The project plan proposed: compute $\bar{d}$, and if it is ≤ 6.0 Å, declare the circuit a real physical site.

The problem is that $\bar{d}$ depends heavily on two things that have nothing to do with whether the circuit is meaningful:

How many residues are in the set. Two random residues are on average closer than ten random residues, simply because averaging more pairs pulls in more long-range ones.

How large the protein is. Three residues drawn at random from a 60-residue domain will often be within 6 Å of one another. The same three drawn from a 600-residue multi-domain protein essentially never will.

So a fixed threshold tests the protein at least as much as the circuit. Apply it to small proteins and everything passes; apply it to large ones and nothing does.

The permutation test

The fix is to compare the circuit against a **null distribution** built from the same protein and the same set size.

1. Compute the observed $\bar{d}$ for the circuit’s K residues.
2. Draw K residues uniformly at random from the structure. Compute their $\bar{d}$.
3. Repeat 10,000 times, building a distribution of what $\bar{d}$ looks like for K arbitrary residues in *this* protein.
4. The p-value is the fraction of random sets at least as compact as the circuit.

```
p_value = float((np.sum(null <= observed) + 1) / (n_permutations + 1))
```

Both size and protein are now controlled, because every null draw has the same K and comes from the same structure. The p-value is a statement about the circuit.

Three details

The +1 correction. The numerator and denominator are each incremented by one. Without it, a circuit more compact than all 10,000 draws would get $p = 0$, which claims more than a finite number of permutations can support. With it, the smallest achievable value is $1/10001$, which is an honest statement of the resolution of the test.

Sampling only resolved residues. The null draws from residues that have coordinates, not from all sequence positions. This matters more than it looks: disordered regions are systematically surface-exposed and spatially spread out, so a null including them would be biased toward looser sets — and would make almost any circuit look significant. Restricting to resolved residues is what keeps the test honest.

One-sided. We ask whether the circuit is *more compact* than chance, not whether it is unusual in either direction. A circuit that is unusually *spread out* is not evidence for the biophysical hypothesis.

What is reported

`SpatialClusteringResult` carries the observed distance, radius of gyration, the null mean and standard deviation, the p-value, a z-score, whether the fixed threshold was met, and the number of unmapped residues. The `verdict` property combines the two criteria:

$p < 0.05$	$\bar{d} \leq 6 \text{ \AA}$	verdict
yes	yes	valid biophysical circuit
yes	no	significantly clustered, but looser than the 6 Å threshold
no	yes	within 6 Å, but no more compact than chance for this size
no	no	spurious / model artefact

The threshold is kept because it is easy to read and the plan calls for it. **The p-value is what carries the argument.**

The test that validates the test

`tests/conftest.py` builds a 40-residue structure with known geometry: an extended α -helix, except that residues 10, 12 and 15 are deliberately placed within about 4 Å of one another, forming a constructed pocket.

Two assertions follow. The planted pocket must come out significant with verdict “valid biophysical circuit”. A spread-out set — residues 1, 14, 27, 40 along the helix — must not. A statistical test that cannot distinguish a planted signal from its absence is not worth running on real data.

6.6 Secondary structure

DSSP (Define Secondary Structure of Proteins) is the standard program for assigning secondary structure from coordinates. It reads a structure and labels each residue: H α -helix, E β -strand, G 3_{10} -helix, T turn, S bend, and so on.

This would let you characterise a circuit beyond its geometry — “the circuit consists of two residues on adjacent β -strands plus one on the intervening loop” — and would let feature purity (Section 4.9) be computed against structural labels rather than amino-acid identity, which is the more interesting axis.

DSSP requires the mkdssp binary, which is not a pip dependency. So `secondary_structure()` returns `None` and warns if it is unavailable, rather than raising:

```
except Exception as exc:
    warnings.warn(
        f"DSSP unavailable ({exc}); install it with `conda install -c salilab dssp` "
        "or `brew install dssp` to annotate secondary structure. "
        "Geometry validation does not depend on it.",
        RuntimeWarning, stacklevel=2,
    )
return None
```

The structural claim rests on the C α geometry. Secondary structure annotates it; it does not support it. An optional enrichment should not be able to break a required analysis.

Part 7 — Benchmarking

7.1 What a good method must do

Having built a method, we have to show it beats the alternatives. The alternatives are the post-hoc saliency methods of Part 3.2, and the comparison has to be on ground that does not beg the question.

In particular, we cannot evaluate by “which heatmap looks more biologically plausible”. That is the failure mode the whole project exists to escape. Both metrics below therefore require **intervening on the model**, not merely ranking residues.

Every method produces a score per residue and is evaluated identically:

Method	What it uses	Cost
Causal SAE patching	interventions on autoencoder features	high
Raw attention	attention weights from the mutated position	free
Integrated gradients	gradients along an interpolation path	moderate
Random	nothing	free

7.2 Raw attention

Take the attention weights with the mutated position as query, average over heads and over layers, and read the result as importance.

```
a = attentions[layer][0]          # [heads, seq, seq]
row = a[:, query_token_pos, :]    # [heads, seq]
reduced = row.mean(dim=0)
```

This is the baseline that most protein-LM interpretability papers actually plot, which is why it is the one implemented, rather than a more sophisticated variant like attention rollout or attention-times-gradient.

One practical note: newer attention implementations (SDPA, FlashAttention) do not expose the weight matrix, because they never materialise it. The model must be loaded with `attn_implementation="eager"` for the weights to be readable, and the benchmark sets this. If it fails, the code substitutes random scores and says so, rather than silently reporting zeros.

7.3 Integrated gradients ▲

The problem with plain gradients

The gradient $\partial m / \partial x$ tells you the *local* sensitivity of the output to an input. That is not the same as how much that input contributed.

The standard counterexample: suppose the model computes $y = \min(x, 1)$ and $x = 3$. The output depends entirely on x — set x to zero and y collapses — but the gradient at $x = 3$ is exactly zero, because the function has saturated. Plain gradients report no importance for an input that determines the answer.

Saturation is everywhere in trained networks (softmaxes, ReLUs past their kink, any near-certain prediction), so this is not a corner case.

The fix

Integrated gradients integrates the gradient along a straight path from a *baseline* input to the actual input, instead of evaluating it at one point:

$$IG_i = (x_i - x'_i) \cdot \int_0^1 \partial m(x' + \alpha(x - x')) / \partial x_i \, d\alpha$$

where x' is the baseline. Along the path the model passes through the non-saturated regime, so the accumulated gradient picks up the contribution that a point evaluation misses.

The integral is approximated by a Riemann sum with steps samples (default 32). The implementation uses **midpoint** sampling:

```
alpha = (step + 0.5) / steps
```

This is strictly more accurate than left endpoints at identical cost, and endpoint sampling error is a large part of what people experience as “IG being noisy”.

Two choices that change the numbers

The baseline. A zero embedding is conventional, but zero is not a meaningful “absence of a residue” for a protein language model — it is a point outside the embedding manifold entirely. The <mask> embedding is the model’s own representation of an unknown residue and is the more defensible reference. Both are implemented (baseline="zero" / "mask"); the benchmark reports the zero baseline so the comparison matches published practice, not because it is better.

The contraction. IG gives an attribution per embedding *dimension*; we need one number per residue. Taking the L2 norm across dimensions discards sign and yields an unsigned magnitude. A signed dot product with the input is the alternative, and it mixes attribution with input scale.

These choices are documented in the code precisely because they are the kind of thing that silently differs between implementations and makes published numbers incomparable.

7.4 Random

Uniform random scores.

This is included because a surprising number of saliency comparisons omit it, and on faithfulness curves in particular random is a much stronger competitor than people expect. Ablating 20% of any protein’s residues damages the model substantially regardless of which 20% you pick. **A method that does not clearly separate from random has demonstrated nothing**, and without the baseline on the plot it is easy not to notice.

7.5 Causal precision

The question: of the residues a method highlights, what fraction actually matter?

The procedure: take the top n residues (default 10). Mutate each one independently to alanine — glycine if it is already alanine, and skip non-canonical residues rather than forcing them. Score each with ESM-2’s masked-marginal score (Section 2.10). Count what fraction move the score by more than a threshold.

```
mutation = alanine_substitution(sequence, seq_pos)
score = wrapper.masked_marginal_score(sequence, seq_pos, mutation.wt_aa, mutation.mut_aa)
effects.append(abs(score))
precision = float(np.mean([e > threshold for e in effects]))
```

Alanine scanning is the right probe because alanine removes the side chain while leaving the backbone intact (Section 1.4), so a large effect implicates the side chain rather than the fold.

A method that highlights residues which do nothing when mutated scores badly here, however convincing its heatmap.

One honest caveat

The probe is the model’s own masked-marginal score, not an experimental assay. So causal precision measures whether the highlighted residues matter **to ESM-2**, not whether they matter in a cell. That is the correct scope — the project explains ESM-2, it does not validate it — but it should not be overstated. Using experimental DMS scores as the probe instead would be a stronger and quite feasible extension.

7.6 Faithfulness

The question: if you remove what a method says is important, how much does the model’s behaviour degrade — compared to removing the same amount at random?

The procedure: for each of several fractions (2%, 5%, 10%, 20%, 30%), mask the top-scoring residues at that fraction and recompute the metric.

Two design points:

Masking, not deletion. Residues are replaced with <mask> rather than removed. This keeps the sequence length fixed, so the mutated position stays where the metric expects it and no positional shift contaminates the measurement.

The mutated site is protected. It is never ablated. Masking it would destroy the readout rather than test the circuit — the metric reads log-probabilities at that position, and masking it changes what question is being asked.

The result is normalised by the clean-versus-corrupted gap, so variants with different effect sizes are comparable, and summarised as an area under the curve.

Reading the curves

The absolute drop is not very informative on its own — it says more about the protein's robustness than about the method. **The gap between a method's curve and the random curve is the quantity of interest.** A method whose curve tracks random has not identified anything, even if its absolute drop is large.

Both panels are written to `figure2_benchmark.png` and `.pdf` by `scripts/benchmark_saliency.py`: causal precision as a bar chart with standard errors, faithfulness as curves. The palette is colour-blind safe and distinguishable in greyscale, since this is headed for a paper.

Part 8 — The codebase, file by file

About 7,300 lines, of which 4,191 are the library under `src/`, 1,360 the runnable scripts and dashboard, and 1,406 the tests. This part is a map. Each entry says what the file is for, what to look at first, and what will bite you.

8.1 `src/utils/` — foundations

`protein.py` (187 lines)

Amino-acid constants, the ESM-2 vocabulary, mutation parsing, and — most importantly — **the index conventions**.

Symbol	Purpose
<code>AA_ALPHABET</code>	the 20 canonical residues
<code>ESM_VOCAB</code>	the 33 ESM-2 tokens in checkpoint order
<code>three_to_one</code>	PDB three-letter codes, with modified residues handled
<code>Mutation</code>	frozen dataclass: <code>wt_aa</code> , <code>seq_pos</code> (0-indexed), <code>mut_aa</code>
<code>parse_mutation</code>	"A45T" → <code>Mutation</code> ; rejects multi-substitutions
<code>seq_to_token_pos</code> / <code>token_to_seq_pos</code>	the only sanctioned index conversions
<code>validate_sequence</code>	normalise and check

Read this file first. Appendix B tabulates the conventions it defines, and essentially every subtle bug in a pipeline of this kind is a violation of one of them.

`Mutation.apply` verifies the wild-type residue matches before substituting, and raises with a message naming the likely cause (a numbering offset between the mutation table and the FASTA) rather than producing a silently wrong sequence.

`device.py` (134 lines)

MPS selection and its consequences.

`resolve_device("auto")` prefers MPS, then CUDA, then CPU. An *explicit* request for an unavailable device raises rather than downgrading — a silent downgrade to CPU on a twelve-hour sweep is the sort of thing you discover the next morning.

`enable_mps_fallback()` sets `PYTORCH_ENABLE_MPS_FALLBACK=1` so a missing Metal kernel degrades to CPU instead of raising. It must be set **before torch is imported**, which is why every entry point does `os.environ.setdefault(...)` at the top of the file, above the imports.

`supports_float64` exists because MPS has no float64 at all. Anything needing doubles — the permutation null, for instance — runs on CPU in numpy.

seeding.py (50 lines)

`seed_everything(seed)` seeds Python, numpy and torch, and returns the seed for logging. Note the honest docstring: full determinism is **not** achievable on MPS, since several kernels are non-deterministic and there is no equivalent of `torch.use_deterministic_algorithms` coverage there. What we can do is seed everything and record the seed, so a run is reproducible up to kernel-level float non-determinism.

`torch_generator(seed)` produces an independent generator for sampling that must not disturb global RNG state — used by the permutation null and the activation shuffle buffer, both of which should be reproducible regardless of how many batches the training loop happened to draw first.

dataloaders.py (581 lines)

Two jobs.

Variant loading. `load_proteingym`, `load_clinvar`, `synthetic_variants`, all normalising to `VariantPair`. `load_variants(cfg)` dispatches on `cfg.source`. Every loader **verifies** that the wild-type residue named in each mutation string is the residue actually present, and drops and counts mismatches.

Activation streaming. `ActivationBuffer` is the piece that makes this run on 8 GB. Read its docstring.

The arithmetic: 10,000 sequences $\times$ ~300 tokens $\times$ 320 dimensions $\times$ 4 bytes $\approx$ 4 GB, alongside a model, on a machine with 8 GB total. So activations are never materialised. The buffer keeps a fixed 256k-token pool, refills from fresh forward passes when it drops below half, and reshuffles the *whole* pool on every refill — not just the new part, or recently added tokens stay clumped by protein of origin.

The shuffle is not cosmetic. Consecutive tokens from one protein are strongly correlated, and an autoencoder trained on unshuffled batches learns per-protein idiosyncrasies that look like features and do not generalise.

`peek(n)` returns tokens without consuming them, for fitting the normaliser and initialising `b_dec` before the first step.

8.2 src/models/ — the model and the dictionary

esm_hooks.py (465 lines)

Residual-stream access. The module docstring explains why the block output is the residual stream under pre-LN.

Symbol	Purpose
<code>resolve_encoder_layers</code>	finds the block list across model classes
<code>HookHandleSet</code>	context manager guaranteeing removal
<code>TokenBatch</code>	tokenised batch plus attention mask
<code>ESMActivationExtractor</code>	capture activations at chosen layers
<code>make_patch_hook</code>	build a hook that rewrites the residual stream
<code>patched_layer</code>	context manager wrapping the above
<code>ESMWrapper</code>	model + tokenizer + derived facts

`ESMActivationExtractor.flat(layer, attention_mask)` returns `[n_real_tokens, d_model]` with padding removed. Dropping padding matters: `<pad>` positions are a large fraction of a batched protein corpus, and they would otherwise dominate the dictionary with one trivial feature.

`ESMWrapper.masked_marginal_score` is the standard ESM-2 variant score, defined here once so no script reimplements it.

The class can be built directly from a model and tokenizer, not only via `from_pretrained` — which is how the tests construct an architecture-identical model with random weights and no network.

`sparse_autoencoder.py` (428 lines)

The Top-K SAE. `SAEConfig` carries architecture **and provenance** (`model_name`, `layer_idx`, `activation_scale`), because a dictionary is only meaningful for the exact layer of the exact checkpoint it was trained on, and loading a layer-3 dictionary against layer-6 activations produces plausible-looking nonsense. `load()` refuses obvious mismatches.

Methods worth knowing:

Method	Purpose
<code>encode</code>	top-k sparsification
<code>decode / decode_sparse</code>	dense and memory-frugal decode
<code>loss</code>	MSE + AuxK, returning metrics
<code>normalize_decoder</code>	project rows back to unit norm
<code>remove_parallel_gradient</code>	strip the radial gradient component
<code>init_b_dec_from_data</code>	initialise the decoder bias to the data mean
<code>save / load</code>	weights plus a JSON config sidecar

The config sidecar is plain JSON deliberately: you can read what a checkpoint is without loading torch.

8.3 `src/interpretability/` — the method

`path_patching.py` (663 lines)

The largest and most important module. Its docstring covers error-preserving splicing, baseline ordering and the sign convention.

Symbol	Purpose
Metric / LogitDiffMetric / SequenceLogLikelihoodMetric	readouts
PatchingSetup	clean/corrupted pair; the only place mutation strings become token positions
PatchResult	one intervention's outcome; dce, normalized_dce
CausalPatcher	the engine
attribution_scores	one-backward-pass approximation
positionwise_effect	per-position DCE for one feature

CausalPatcher methods: `baselines` (run first, no hooks live), `sae_latents`, `active_features` (exact pruning), `direct_causal_effect`, `sweep_features`, `ablate_features`, and the private `_patched` context manager and `_build_transform`.

circuit_extraction.py (474 lines)

Assembling interventions into a circuit. `CircuitNode`, `CircuitEdge`, `Circuit` (JSON-serialisable with full provenance), `greedy_select`, `discover_edges`, `discover_circuit`.

`discover_circuit` runs the five stages: `prune` → `attribution rank` → `exact greedy confirm` → `positional localisation` → `edge discovery`.

`Circuit.recovered_fraction` is the headline number. `_joint_patch_metric` registers all layers' hooks simultaneously via an `ExitStack`.

baselines.py (169 lines)

`attention_saliency`, `integrated_gradients_saliency`, `random_saliency`, `sae_circuit_saliency`. All return scores over residue positions with special tokens stripped, so the evaluation never reasons about token offsets.

8.4 src/validation/ — checking the answer

pdb_aligner.py (546 lines)

`PDBStructure`, `load_structure`, `StructureAlignment`, `align_sequence_to_structure`, `ca_distance_matrix`, `mean_pairwise_distance`, `radius_of_gyration`, `SpatialClusteringResult`, `spatial_clustering_test`, `evaluate_circuit_geometry`, `secondary_structure`.

`evaluate_circuit_geometry` is the one-call entry point: `align`, `map`, `test`.

`load_structure` deliberately does **not** download. Fetching lives in `scripts/fetch_assets.py` so an analysis run is reproducible offline and cannot quietly depend on network state.

monosemanticity_metrics.py (394 lines)

evaluate_reconstruction (FVU, L0, dead fraction, cosine), cross_entropy_recovered (the behavioural metric), profile_features, feature_purity, normalized_entropy, summarise_profiles.

profile_features maintains a running top-k merge across batches, so corpus size is not bounded by memory.

8.5 scripts/

Script	Interface	What it does
smoke_test.py	none	full pipeline on a tiny random-weight model; no downloads
fetch_assets.py	argparse	every download in the project
train_sae.py	Hydra	one SAE per target layer
discover_circuits.py	argparse	the single-command demo
benchmark_saliency.py	Hydra	causal precision + faithfulness

The interface split is deliberate. `discover_circuits.py` is run by hand with a PDB ID and a mutation, and `--mutation A45T` reads better than `+mutation=A45T`. The two scripts that get swept over configurations use Hydra.

8.6 dashboard/app.py (372 lines)

Streamlit plus py3Dmol. Loads saved circuit JSON, renders residues on the structure coloured by effect size, with the mutated site in a distinct colour.

It reads *saved* circuits rather than running discovery live, because discovery takes minutes and needs the model in memory, and Streamlit re-runs the whole script on every widget interaction. A dashboard that re-ran discovery on every slider move would be unusable.

8.7 tests/ (1,406 lines, 79 tests)

No network. No pretrained weights. `conftest.py` builds a real `EsmForMaskedLM` from `EsmConfig` at toy dimensions (32-wide, 4 layers) with the real 33-token vocabulary written to disk, so the actual transformers module graph is exercised — layer resolution, hook placement, output-tuple handling, token offsets all run against the real ESM-2 code path.

`mini_pdb` generates a valid fixed-column PDB file with known geometry.

Part 9 — Running it

9.1 Install

```
conda create -n causalmech python=3.10 -y
conda activate causalmech

cd "Causal Mechanistic Circuit Discovery"
pip install -r requirements.txt
pip install -e .
```

On Apple Silicon, the default PyPI torch wheel is already the arm64 build with MPS support. Do **not** install a +cpu or +cu* variant.

`pip install -e .` installs the project in editable mode so that `src/` is on the import path; this is what lets scripts do `from models.esm_hooks import ...`

9.2 Verify the install — 1 minute, no downloads

```
python scripts/smoke_test.py
```

Expected output, roughly:

```
[smoke] device=mps mps_fallback=1 float64=unsupported torch=2.x
[smoke] 1/5 model: 4 layers, d_model=64
[smoke] 2/5 activations: (1526, 64)
[smoke] 3/5 SAE: L2 FVU=0.0112 EV=0.9888 L0=8.0 dead=0.0% cos=0.9968
[smoke] 4/5 variant H108T: clean=0.1788 mutant=-0.1568
[smoke]      3 nodes, 1 residues, joint recovery 76.0%
[smoke] 5/5 geometry: too few residues for a clustering test (fine here)
[smoke] PASS — pipeline is wired correctly.
```

This proves every module imports, the shapes line up and the stages compose. It proves nothing biological — the weights are random, so there is nothing real to find. Run it before spending an hour on downloads.

Then the test suite:

```
python -m pytest          # 79 tests, also no downloads
```

9.3 Fetch assets — 30-90 minutes, depending on connection

```
python scripts/fetch_assets.py --all
```

Or selectively:

```
python scripts/fetch_assets.py --models
python scripts/fetch_assets.py --pdb 1A2Y 4HHB 1UBQ
python scripts/fetch_assets.py --corpus --corpus-size 10000
python scripts/fetch_assets.py --proteingym # large, several hundred MB
```

If ProteinGym 404s, its release URLs move between versions; check <https://proteingym.org> for the current link.

Everything lands under assets/, which is gitignored. After this, the rest works offline.

9.4 Train the autoencoders — 15-20 min per layer (8M model)

```
python scripts/train_sae.py
```

One SAE per layer in `model.target_layers` (default [2, 3, 4]). Useful overrides:

```
python scripts/train_sae.py model=esm2_35m
python scripts/train_sae.py model.target_layers=[3]
python scripts/train_sae.py sae.arch.k=64 sae.arch.dict_mult=32
python scripts/train_sae.py sae.train.total_steps=50000
```

Output goes to a timestamped Hydra directory under outputs/, containing `sae_layer{N}.pt`, its `.json` sidecar, `training_summary.json`, and the config actually used.

What to watch during training. The progress bar shows `fvu` and `dead`. FVU should fall steadily; dead fraction should stay near zero. If dead fraction climbs past 20% and stays there, AuxK is not doing its job — raise `aux_k` or lower `dead_after_tokens`.

Table 1 is printed at the end:

layer	FVU	EV	L0	dead	cos
2	0.0834	0.9166	32.0	0.4%	0.9712
3	0.0921	0.9079	32.0	0.7%	0.9688
4	0.1104	0.8896	32.0	1.1%	0.9601

(Illustrative shape, not measured values — your numbers will differ.)

9.5 Discover a circuit — 30-60 seconds per variant

```
python scripts/discover_circuits.py --pdb 1A2Y --mutation A45T --layer 3
```

Other input modes:

```
python scripts/discover_circuits.py --sequence MKTAYIAK... --mutation H64A --layers 2 3 4
python scripts/discover_circuits.py --fasta my_protein.fasta --mutation L67A --layer 3
```

Useful flags:

Flag	Effect
--no-edges	skip edge discovery (faster)
--splice-mode replace	naive splicing, for comparison
--permutations 50000	tighter p-value resolution
--max-features-per-layer 6	allow larger circuits
--position-threshold 0.05	looser residue selection
--sae-dir path/to/dir	use a specific SAE run instead of the newest

A common first error

mutation A45T expects A at position 45 but the sequence has L.

Note: with --pdb the sequence is the **resolved** chain, so its numbering starts at the first residue with coordinates and may not match the author numbering in the PDB file.

This is the Section 6.2 problem announcing itself. With --pdb, the wild-type sequence is built from residues that *have coordinates*, so its numbering starts at 1 regardless of what the PDB file calls them. If your mutation uses author numbering, pass the sequence explicitly with --sequence.

The check is deliberate. Proceeding would analyse a different mutation than the one you asked for.

9.6 Benchmark — ~20 minutes for 25 variants

```
python scripts/benchmark_saliency.py
python scripts/benchmark_saliency.py data.source=synthetic +n_variants=50
python scripts/benchmark_saliency.py model=esm2_35m +top_n_residues=15
```

Writes benchmark.json and figure2_benchmark.png / .pdf to the run directory, and prints a summary table.

9.7 Inspect — interactive

```
python -m streamlit run dashboard/app.py
```

Opens in a browser. It finds circuit JSON files automatically under outputs/circuits/.

9.8 A complete first session

```
conda activate causalmech
python scripts/smoke_test.py # 1 min
python -m pytest # 2 min
python scripts/fetch_assets.py --models --pdb 1UBQ --corpus --corpus-size 5000
python scripts/train_sae.py model.target_layers=[3] # ~20 min
```

```
python scripts/discover_circuits.py --pdb 1UBQ --mutation L67A --layer 3
python -m streamlit run dashboard/app.py
```

1UBQ is ubiquitin — 76 residues, extremely well characterised, and small enough that the whole loop is fast. A good first target.

Part 10 — Reading the results

Having produced numbers, the question is whether to believe them. This part is a checklist, roughly in order of how often each thing goes wrong.

10.1 Was there an effect to explain?

```
[circuits] metric: clean=2.1043 mutant=0.3319 gap=1.7724
```

Check the gap first. If it is below about 0.1, the model barely responded to the mutation, and everything downstream is a decomposition of noise. Normalised DCE values will be large and meaningless because the denominator is tiny.

The script warns below $1e-3$. Use judgement above that. Pick a variant the model actually responds to, or accept that the result is about a non-response.

10.2 Does the circuit hold together?

```
[circuits] 4 nodes, 3 edges
```

```
[circuits] joint recovery: 71.3% of the mutation's effect
```

Joint recovery is the headline number. Rough reading:

Joint recovery	Interpretation
> 60%	a substantive circuit
30-60%	a real but partial account; more features or more layers may be needed
< 30%	the mechanism is mostly elsewhere — a different layer, or not captured by the dictionary
> 100%	overshoot; possible, and worth investigating rather than celebrating

Compare it against the individual node effects printed below. If several nodes each show 60% individually but the joint recovery is also 60%, they are redundant — encoding the same thing — and you have one mechanism, not four. Greedy selection is supposed to prevent this, but it is worth confirming.

10.3 Is the circuit spatially real?

```
[circuits] structure alignment: 1UBQ:A identity=100.0% coverage=100.0% aligned=76/76
[circuits] mean pairwise Cα distance: 7.12 Å (random same-size sets: 14.83 ± 3.21 Å)
[circuits] p=0.0021 z=-2.40 Rg=4.55 Å
[circuits] verdict: significantly clustered, but looser than the 6 Å threshold
```

Check the alignment line before the geometry line. Identity below about 80% means you are probably looking at the wrong structure or the wrong chain, and nothing below it is interpretable. Low coverage means much of your sequence has no coordinates.

Then the p-value. The verdict string combines both criteria (Section 6.5), and the four possible verdicts mean different things:

- **valid biophysical circuit** — significant *and* within 6 Å. The strong result.
- **significantly clustered, but looser than 6 Å** — real clustering at a larger scale than a single active site. A domain interface, perhaps. Still a finding.
- **within 6 Å, but no more compact than chance** — the threshold was met because the protein is small. Not evidence.
- **spurious / model artefact** — no spatial structure.

Watch for `n_unmapped`. If four of your seven circuit residues had no coordinates, the geometry test ran on three, and its conclusion is correspondingly thin.

10.4 Is it better than the free alternatives?

From the benchmark:

method	precision	AUC(drop)	n
causal_sae	0.720 ± 0.041	0.612	23
attention	0.430 ± 0.055	0.388	23
integrated_gradients	0.510 ± 0.049	0.451	23
random	0.260 ± 0.038	0.301	23

(Illustrative shape, not measured values.)

Read the random row first. If a method does not clearly separate from random, it has demonstrated nothing, whatever its absolute numbers. The standard errors matter: with 25 variants, a gap of 0.05 is not a result.

10.5 Warning signs

A checklist of things that mean stop and investigate.

Every DCE is exactly zero. Almost certainly the hook-leak bug of Section 5.8, or an SAE whose features never fire. Run `smoke_test.py`; it has an explicit assertion for the no-op control.

Dead fraction above 50% after training. Most of your dictionary does not exist. Everything built on it is running on a fraction of the intended capacity.

FVU above 0.3. The autoencoder is not fitting. Patching in a basis that does not reconstruct the activations is not meaningful.

Normalised DCE far above 1.0 with a tiny gap. Denominator artefact (Section 5.5). Not a result.

The circuit is exactly the mutated residue and nothing else. Suspicious. The metric reads out at that position, so a feature acting there can influence it trivially. Interesting circuits involve *other* residues.

Every variant gives the same features. Those features are probably encoding something generic — position, a token-identity signal, an artefact — rather than anything about the specific mutation.

Alignment identity around 30%. You are aligning two unrelated proteins. Check the PDB ID and the chain.

10.6 The comparison worth running once

```
python scripts/discover_circuits.py --pdb 1UBQ --mutation L67A --layer 3
python scripts/discover_circuits.py --pdb 1UBQ --mutation L67A --layer 3 --splice-mode ↔
replace
```

The second uses naive splicing. Comparing the two shows, on your own data, how much of a naive analysis is measuring the autoencoder’s reconstruction error rather than anything about the model. It is the most convincing demonstration of Section 5.7 available, and it takes two minutes.

Part 11 — Limitations and what to do next

11.1 Limitations

Stated plainly, because a method's limits are part of the method.

Circuits are per-variant. Nothing here establishes that a circuit found for one mutation generalises to other mutations in the same protein, let alone across proteins. Every result is a case study. **This is the most important gap.**

Single substitutions only. `parse_mutation` rejects multi-mutants explicitly, because single-site causal attribution is not well defined for a joint mutant. Much of ProteinGym is multi-mutant and is therefore unused.

The metric is the model's opinion, not an assay. `LogitDiffMetric` reads ESM-2's log-probabilities. It correlates with function, but the project explains ESM-2 — it does not validate ESM-2 against biology.

Attribution prescreening can silently miss features. The shortlist comes from a first-order approximation that is unreliable where the response saturates (Section 5.10). A feature invisible to the gradient never reaches exact patching. `top_k_attribution` controls the slack; nothing detects the miss.

Denoising only. The project patches clean into corrupted (sufficiency) and does not run the reverse (necessity). Redundant mechanisms are therefore systematically over-reported relative to what a necessity analysis would find.

One layer's SAE at a time for edges. Edge discovery measures how an upstream feature changes a downstream feature's activation, not how it changes the downstream feature's *effect*. A full treatment would patch along paths rather than at nodes.

Dictionaries are checkpoint- and corpus-specific. `load()` refuses obvious mismatches, but nothing can detect a dictionary trained on a meaningfully different corpus from the one you are analysing.

Alanine scanning is one probe. Causal precision uses alanine substitution. A residue whose importance is about its *size* rather than its chemistry may score low when the probe is "remove the side chain".

No structural confound control. Buried residues are both more likely to be functionally important and more likely to be near other residues. The permutation null controls for set size and protein size but not for burial. A circuit could score significant partly because it selects buried residues for reasons unrelated to the mutation. Sampling the null from residues matched on solvent accessibility would tighten this considerably, and is the first thing I would add to Part 6.

11.2 What to do next, in order

- 1. Cross-variant generalisation.** Run discovery for every variant in one ProteinGym assay and ask whether the same features recur. If a small set of features accounts for many variants, that is a mechanism rather than a case study, and it is the finding that would carry a paper. Concretely: build a variant $\times$ feature matrix of normalised DCE, and look at its rank.
- 2. A burial-matched null.** Compute relative solvent accessibility (DSSP gives it) and sample the permutation null from residues matched on it. This removes the confound above and makes the structural claim considerably stronger.
- 3. Label features by structure, not identity.** Feature purity is currently computed over amino-acid identity, and Section 4.9 explains why that is the least interesting axis. Recompute it over DSSP secondary structure and burial. This is a small change to `profile_features` and would likely be the most informative single addition to the feature analysis.
- 4. Necessity as well as sufficiency.** Add the noising direction — patch corrupted values into the clean run — and report both. Where they disagree, you have found redundancy, which is itself interesting.
- 5. Use experimental effects as the probe.** Replace the masked-marginal score in causal precision with the measured DMS score. This changes the claim from “these residues matter to ESM-2” to “these residues matter in the assay”, which is a much stronger statement.
- 6. Scale to 35M and check stability.** Run the same variants on both checkpoints. Features will not correspond one-to-one, but if the *residues* a circuit selects are stable across model scale, that is evidence the finding is about proteins rather than about one model.
- 7. Then write the paper.** `paper/README.md` lists which figures and tables the pipeline already produces. Item 1 is what the results section needs.

Appendix A — Glossary

Ablation. Removing a component (setting it to zero or to its mean) to see how much the output degrades. Complements patching.

Activation. The intermediate values a network computes for a particular input, as opposed to the weights, which are fixed.

Activation patching. Replacing an internal value during a forward pass with the value it took on a different input. The core experimental method here.

Active site. The residues of an enzyme that perform its chemical reaction; typically a handful, brought together by folding from scattered sequence positions.

Adam. An optimiser that scales each parameter’s step by running estimates of the gradient’s first and second moments.

Affine gap penalty. A gap-scoring scheme charging a large cost to open a gap and a small cost to extend it. Encodes the belief that long gaps come from single deletion events.

Alanine scanning. Mutating residues to alanine to remove side chains while leaving the backbone intact. The functional probe used in the benchmark.

Amino acid. The building block of proteins. Twenty are commonly used.

Ångström (Å). 10^{-10} metres. A hydrogen bond is about 3 Å; residues in contact are within about 6–8 Å.

Attention. The transformer mechanism by which one sequence position reads from others, using query, key and value vectors.

Attribution patching. Approximating every feature’s causal effect with one backward pass via a first-order Taylor expansion. Used for ranking only.

Autoencoder. A network trained to reproduce its input through a constrained intermediate representation.

AuxK. An auxiliary loss asking dead latents to reconstruct the main reconstruction’s error, which revives them.

Backpropagation. The algorithm computing gradients through a network by applying the chain rule backwards.

BLOSUM62. A substitution matrix scoring amino-acid replacements by how often they occur in related proteins. Used in sequence alignment.

C α (alpha carbon). The central backbone carbon of a residue; the conventional single-point representative of its position.

Circuit. A small set of features, at specific layers and positions, whose joint restoration recovers most of a behaviour.

ClinVar. A public database of human genetic variants annotated with clinical significance.

Corrupted. The perturbed input in a patching experiment — here, the mutant sequence.

Cross-entropy. The standard loss for predicting a category; $-\log$ of the probability assigned to the correct answer.

Cross-entropy recovered. How much of a model’s language-modelling performance survives when its activations are replaced by an autoencoder’s reconstruction. The behavioural measure of dictionary quality.

DCE (direct causal effect). The change in the metric caused by one patch.

Dead latent. An autoencoder feature that has stopped firing and therefore stopped receiving gradient.

Deep mutational scanning (DMS). An experiment measuring the functional effect of many mutations at once.

Denoising. Patching clean values into a corrupted run; tests sufficiency.

Dictionary learning. Recovering a set of directions such that observations are sparse combinations of them.

DSSP. The standard program for assigning secondary structure from coordinates.

Embedding. A learned vector representing a token.

ESM-2. Meta’s protein language model family, trained on UniRef by masked language modelling. The subject of this project.

Error-preserving splicing. Applying a patch by *adding the difference* of two decodes rather than substituting one, so the autoencoder’s reconstruction error cancels exactly.

Faithfulness. Whether an explanation describes what the model actually did. Measured here by ablating the residues a method selects.

Feature. A direction in activation space corresponding to an interpretable property.

Forward hook. A PyTorch callback firing after a module computes its output, able to inspect or replace it.

FVU (fraction of variance unexplained). Reconstruction error divided by total variance. The scale-free reconstruction metric.

Gradient. The derivative of the loss with respect to a quantity.

Hydrophobic. Water-avoiding. Hydrophobic side chains tend to be buried in a folded protein’s core.

Integrated gradients. A saliency method integrating the gradient along a path from a baseline to the input, which fixes plain gradients’ saturation problem.

Joint recovery. The normalised effect of patching all a circuit’s features together in one forward pass. The headline number.

LayerNorm. Normalising a vector to zero mean and unit variance across features, then applying a learned scale and shift.

Linear representation hypothesis. The assumption that models represent interpretable properties as directions in activation space.

Logit. A raw, unnormalised score before softmax.

Logit difference. The difference of two logits at one position. Shift-invariant, which is why it is the metric of choice.

L0. The count of non-zero entries in a sparse code. Fixed to k by construction in a Top-K autoencoder.

L1 penalty. A sparsity-inducing penalty on the sum of absolute values. Causes shrinkage and needs a tuned coefficient; avoided here.

Masked language model. A model trained to predict hidden tokens from context on both sides. ESM-2 is one.

Monosemantic. Corresponding to exactly one interpretable property. The opposite of polysemantic.

MPS. Metal Performance Shaders — PyTorch’s GPU backend on Apple Silicon.

Mutation. A change to a protein’s sequence. Written A45T: wild-type A at 1-indexed position 45, replaced by T.

Noising. Patching corrupted values into a clean run; tests necessity.

Normalised DCE. DCE divided by the full clean-minus-corrupted gap. 1.0 means the patch explains the whole effect.

PDB (Protein Data Bank). The public archive of experimentally determined protein structures.

Permutation test. Building a null distribution by random resampling and computing an empirical p-value against it.

Polysemantic. Responding to several unrelated things. The normal state of individual neurons.

Post-hoc. Applied to a trained model after the fact, by observation rather than intervention.

Pre-LN. The transformer variant applying LayerNorm to a sub-layer’s input rather than its output. ESM-2 uses it, which is what makes the residual stream directly readable and writable.

ProteinGym. A curated benchmark of deep mutational scanning assays.

Protein language model. A neural network trained on protein sequences with a language-modelling objective.

ReLU. $\max(0, x)$. The standard non-linearity.

Residual stream. The running vector at each position that every transformer sub-layer reads from and adds to. The model’s communication channel, and the object this project studies.

Residue. One amino acid within a protein chain.

Rotary position embedding (RoPE). Encoding position by rotating query and key vectors, so attention depends on relative separation. Used by ESM-2.

Saliency. A score per input element, usually visualised as a heatmap.

Secondary structure. Local repeating shape — α -helix, β -sheet, loop.

Sparse autoencoder (SAE). An overcomplete autoencoder with a sparsity constraint, used to recover a model’s feature directions.

Superposition. Representing more features than there are dimensions, by assigning features almost-orthogonal directions and relying on sparsity to limit interference.

Top-K. Enforcing sparsity by keeping the k largest activations and zeroing the rest, rather than by penalising magnitude.

Wild type. The reference, unmutated sequence. The control condition.

Appendix B — Index conventions

The most common source of silent error in a pipeline like this. All conversions happen in `src/utils/protein.py` and nowhere else.

Quantity	Convention	Example for A45T
Mutation string position	1-indexed	45
<code>Mutation.one_indexed_pos</code>	1-indexed	45
<code>Mutation.seq_pos</code>	0-indexed into the sequence string	44
Token position	0-indexed, offset by <cls>	45
<code>CircuitNode.token_positions</code>	0-indexed token positions	[45, 48]
<code>CircuitNode.residues</code>	1-indexed residue numbers	[45, 48]
PDB residue numbers	author numbering, arbitrary	may be anything
<code>StructureAlignment.seq_to_token</code>	1-indexed → 0-indexed row of <code>ca_coords</code>	

Rules:

- `seq_to_token_pos` and `token_to_seq_pos` are the only sanctioned conversions.
- `token_to_seq_pos(0)` **raises** — `<cls>` is not a residue.
- Anything user-facing (printed, saved to JSON, shown in the dashboard) is **1-indexed**, because that is what biology uses.
- Anything internal is 0-indexed, because that is what Python uses.
- PDB numbering is never assumed to match anything. It is always resolved through the alignment.

Appendix C — Notation

Symbol	Meaning
$d_{\text{model}}, d_{\text{in}}$	residual stream width (320 for ESM-2 8M)
d_{sae}	dictionary size ($d_{\text{in}} \times \text{dict_mult}$, 5120 by default)
k	active latents per token (32 by default)
x	an activation vector, $[n_{\text{tokens}}, d_{\text{in}}]$
z	a sparse code, $[n_{\text{tokens}}, d_{\text{sae}}]$
W_{enc}	encoder, $[d_{\text{in}}, d_{\text{sae}}]$
W_{dec}	decoder, $[d_{\text{sae}}, d_{\text{in}}]$; row j is feature j's direction
$b_{\text{enc}}, b_{\text{dec}}$	encoder and decoder biases
$a_{\text{clean}}, a_{\text{corrupt}}$	residual stream on the two inputs
ϵ	autoencoder reconstruction error
m	the metric
$\bar{d}$	mean pairwise C α distance of a residue set
K	number of residues in a circuit

End of guide.